

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

Yesterday's Ark

Path Planning and Tracking Control for an Autonomous Marine Vehicle

Intelligent Ship Innovation Practice

Assignment 4

School Ocean Engineering

Class F2101002

Authors Xue Jiang, Linyu Li, Ruoxuan Wang

Lixing Rui, Yumeng Du, Runtao Zhang

September 9, 2026

Contents

1 Application Background Analysis **3**
- 1.1 The Connotation of Ship Intelligence . . . 3
- 1.2 Current Development Status of Intelligent Ships at Home and Abroad . . . 3
  - 1.2.1 Domestic Development Status of Intelligent Ships . . . 4
  - 1.2.2 Development Status of Intelligent Ships Abroad . . . 5

2 Methods and Models **7**
- 2.1 Sampling-Based Path Planning Algorithms . . . 7
  - 2.1.1 Basic Concepts . . . 7
  - 2.1.2 The RRT Algorithm . . . 7
- 2.2 Implementation and Optimisation of the Algorithm . . . 8
  - 2.2.1 Collision-Checking Algorithm . . . 9
  - 2.2.2 Safety-Radius Optimisation . . . 9
  - 2.2.3 Turning-Angle Constraint Optimisation . . . 9
  - 2.2.4 Probability-Concentration Optimisation . . . 9
  - 2.2.5 Path-Pruning Optimisation . . . 9
  - 2.2.6 Evaluation-Criterion (Multiple-Iteration) Optimisation . . . 10
  - 2.2.7 Code-Structure Optimisation . . . 11
- 2.3 Introduction to the LOS Guidance Algorithm . . . 12
  - 2.3.1 Obtaining the Heading Angle . . . 12
  - 2.3.2 Analysis of the LOS Navigation Modes . . . 13
  - 2.3.3 Design of the LOS-Based PID Controller . . . 14
- 2.4 Optimisation of the Path-Tracking Code . . . 15
  - 2.4.1 The PID Function . . . 15
  - 2.4.2 Correction of the PID Control Angle . . . 16
  - 2.4.3 Throttle Control and the Corresponding Adjustment of the PID Parameters 17

3 Experimental Results and Analysis **19**
- 3.1 Results and Optimisation of the Planning Algorithm . . . 19
  - 3.1.1 Evaluation Metrics . . . 19
  - 3.1.2 Visual Results of the Individual Optimisations . . . 21
    - 3.1.2.1 Optimisations Applied Separately . . . 21
    - 3.1.2.2 Optimisations Applied in Combination . . . 23
    - 3.1.2.3 Combined Optimisation . . . 25
  - 3.1.3 Change of the Evaluation Metrics after Each Optimisation . . . 26
    - 3.1.3.1 Optimisations Applied Separately . . . 26
    - 3.1.3.2 Optimisations Applied in Combination . . . 27
    - 3.1.3.3 Combined Optimisation . . . 28
- 3.2 Results and Optimisation of the Control Algorithm . . . 30
  - 3.2.1 Evaluation Metrics . . . 30
  - 3.2.2 Change of the Evaluation Metrics after Each Optimisation . . . 31
    - 3.2.2.1 Results of the Control Algorithm without Throttle Control . . . 32
    - 3.2.2.2 Results of the Control Algorithm with Throttle Control . . . 37

4 Remaining Problems **38**
4.1 Problems of the Planning Algorithm 38
4.2 Problems of the Control Algorithm 38
4.2.1 Problems of the Throttle Control 38
4.2.2 Problems of the Data of the Control Group 38
4.2.3 Problems of the Path of the Planning Algorithm 38

5 Conclusions and Outlook **39**
5.1 Summary 39
5.2 Outlook 39
5.2.1 Adding a Perception Module 39
5.2.2 Adding Collision-Avoidance Rules 39
5.2.3 On the Throttle Control 39

6 Appendix **42**

1 Application Background Analysis

1.1 The Connotation of Ship Intelligence

The term “intelligent ship” is widely used in domestic research, and related concepts include “unmanned ship”, “smart ship”, “remotely controlled ship” and “autonomous ship”. The designation currently accepted by the international community is “Maritime Autonomous Surface Ship” (MASS), sometimes shortened to “autonomous ship”.[1]

According to the 2020 edition of the *Rules for Intelligent Ships* issued by the China Classification Society (CCS), ship intelligence is built on advanced sensing technology, data processing and analysis technology, automatic control technology, communication technology and artificial intelligence technology. It enables ship decision support, remote control and unmanned autonomous control functions, so that the operation of a ship throughout its life cycle becomes safer, more economical and more efficient.

In the civil sector such vessels are usually called intelligent ships, whereas in the military sector they are usually called unmanned vessels, as shown in Figure 1.1.



(a) Civil vessel (b) Naval vessel

Figure 1.1: Intelligent ships and unmanned vessels

1.2 Current Development Status of Intelligent Ships at Home and Abroad

In May 2006 the International Maritime Organization (IMO) adopted the E-Navigation work programme, which is regarded as the earliest concept of the intelligent ship: electronic information technology is used to coordinate ship-shore communication and navigation information, to realise information exchange among the ship, the ship-shore link and the shore side, and to promote marine environmental protection and efficient, safe navigation. Intelligent ships have been listed by the IMO and the International Organization for Standardization (ISO) as an important topic, and the international shipbuilding and shipping industry is bound to move towards ship intelligence. The major classification societies of the world have also issued relevant rules and guidance documents for intelligent ships.[2]

The development stages of intelligent ships as defined by the IMO are as follows:

1) **Ships with automated systems and decision support**
   Limited to remote monitoring of the status of shipborne equipment and to data analysis;

2) **Remotely controlled ships with seafarers on board**
   Technologies such as cloud computing, the Internet of Things and big data analysis are used to provide the ship with periodic safety, environmental protection and energy-efficiency optimisation advice through a connection to a shore centre, realising semi-automatic navigation;

3) **Ships without seafarers on board**
   On the basis of ship data analysis, port logistics information is added to realise a seamless ship-shore information link and to optimise in real time and dynamically the voyage, the schedule and port operations;

4) **Fully autonomous ships**
   Fully autonomous unmanned navigation together with automated port loading, unloading and logistics.

At present, intelligent ships are in the transition stage from the first stage to the second stage.

1.2.1 Domestic Development Status of Intelligent Ships

The intelligent navigation system is one of the key core technologies that China urgently needs to break through, and domestic research on intelligent navigation technology, remote control driving technology and autonomous collision-avoidance technology has been carried out around intelligent navigation systems.

Intelligent ships in China are developed along the path of "decision support – remote control – autonomous navigation".

In 2015, CCS released the *Rules for Intelligent Ships*, the first complete set of rules for intelligent ships in the world, which specifies the technical requirements for intelligent ships in terms of intelligent navigation, intelligent hull, intelligent engine room, intelligent energy-efficiency management, intelligent cargo management and intelligent integration platforms. In 2018, the Ministry of Industry and Information Technology launched the high-technology ship research project, the Intelligent Ship 1.0 programme, to carry out the research and development of intelligent technologies for bulk carriers, tankers and container ships, and the demonstration ships obtained intelligent ship notations such as intelligent navigation, energy efficiency and engine room[3]. In 2019, the Ministry of Science and Technology launched a National Key R&D Programme project, "Research on Key Technologies of Intelligent Navigation and Control of Ships Based on Ship-Shore Collaboration"; the project covers shore-based information support, communication and network security and remote driving technology that support intelligent ship navigation, and develops intelligent navigation systems for key navigable sections of inland ships as well as intelligent route optimisation and autonomous navigation systems for coastal ships.[4]

The research team of Wuhan University of Technology created the "Navigation Brain" intelligent navigation system and designed an "artificial intelligence brain" system that imitates the working mode of the "human brain". The system is designed as three mutually related working space modules, namely the "perception space", the "cognition space" and the "decision and execution space". The three modules cooperate with each other and realise the whole process of "perception, cognition, decision and control" of intelligent ship navigation.

In December 2017, "Dazhi", the world's first intelligent bulk carrier independently developed in China and certified by both CCS and Lloyd's Register, was officially delivered in Shanghai. The ship is equipped with an intelligent navigation system and has the function of navigation decision support.

In November 2018, "Mingyuan", the first large ore carrier demonstration ship of China's Intelligent Ship 1.0 programme, was delivered. The ship obtained certification from CCS and Det Norske Veritas respectively and has the function of assisted autonomous navigation.[5]



Figure 1.2: The large ore carrier “Mingyuan” demonstrating assisted autonomous navigation in 2018

In 2019, Wuhan University of Technology realised an 8 500 km intercontinental remote control test for the first time: a self-propelled model ship navigating in Tangxun Lake in Wuhan, China, was remotely controlled from Wageningen, the Netherlands.

In May 2019, the world’s first intelligent container ship, “Hehua”, was delivered and put into operation. The intelligent ship system of Hehua was independently developed by the Shanghai Ship and Shipping Research Institute together with CCS and other units. Its intelligent navigation system can sense the navigation situation, intelligently switch the ship-shore communication route and build a ship-shore collaborative route selection mode, so as to realise the comprehensive optimisation of navigation risk, timeliness, cost and energy consumption and to guarantee navigation safety.

In May 2019, “Zhiteng”, the first small test ship with an unmanned autonomous navigation system in China, completed its navigation test and initially possessed the functions of autonomous collision avoidance, underwater collision avoidance during autonomous navigation, autonomous berthing and unberthing, autonomous path tracking and autonomous navigation control.[2]

In June 2019, “Kaizheng”, the first very large crude carrier demonstration ship of China’s Intelligent Ship 1.0 research programme, was delivered. The ship is equipped with an electro-optical tracking system and an assisted collision-avoidance system and has the function of assisted autonomous navigation.

1.2.2 Development Status of Intelligent Ships Abroad

The IMO added the topic of “autonomous unmanned ships” in 2017 and approved the *Interim Guidelines for MASS Trials* by circular in 2019. At present, owing to the limitations of international legal conventions and the applicability of intelligent navigation technology, most foreign research on intelligent navigation systems tends to carry out technology research on small ships, such as ship situation awareness, remote control driving, autonomous route optimisation and assisted collision avoidance in open waters.

In 2017, Rolls-Royce cooperated with the global tug operator Svitzer and successfully demonstrated a remote control system for tugs in the port of Copenhagen, Denmark. The 28 m long tug Svitzer Hermod safely performed manoeuvres such as docking, unberthing and 360° rotation, all of which were remotely controlled by the captain at the Svitzer headquarters; the remote operation centre redefined the way the ship is controlled[6].

In 2017, the Norwegian companies YARA and Kongsberg jointly built the first autonomous electric container ship, YARA Birkeland. Through equipment such as the global positioning system, radar, cameras and sensors, the ship is able to avoid obstacles automatically in the

fairway, and the associated automatic mooring system can complete the berthing and departure of the autonomous ship.

In 2018, Rolls-Royce and the Finnish state-owned ferry operator Finferries successfully demonstrated the world's first fully automatic ferry, the car ferry Falco, in the archipelago south of Turku, Finland. The ferry is 53.8 m long and can either operate fully autonomously or be remotely controlled from a command centre. In the same year, the Finnish group Wärtsilä successfully tested an automatic berthing system for ships: witnessed by the Norwegian Maritime Authority, the 85 m long ferry Folgefonn operated autonomously and achieved uninterrupted operation on a route serving three ports. This was the first time that a ship of this size attempted fully automated shore-to-shore navigation in a completely unmanned operation mode.

In January 2019, the ice-class passenger ferry Suomenlinna II successfully crossed the test area near the port of Helsinki under remote control from a control centre and passed the remote sea trial.[2]

Overall, autonomous ship navigation is still in its infancy. Remote control and autonomous navigation have only been explored and functionally tested on small ferries, tugs, test ships and training ships, while large merchant ships remain at the stage of decision support.

2 Methods and Models

2.1 Sampling-Based Path Planning Algorithms

2.1.1 Basic Concepts

A sampling-based path planning algorithm is a method for solving path planning problems: it generates candidate paths by randomly sampling the state space, so as to find a feasible path from the start point to the goal point. The core idea is to explore the state space through random sampling in order to find a path that satisfies the constraints. The main advantage of this approach is its wide applicability: it can handle high-dimensional state spaces and does not require an explicit model of the state space, since the feasibility of a path is verified through collision checking.

Sampling-based path planning algorithms are usually divided into two categories:

1) **Single-query algorithms**

   Single-query algorithms focus on finding a feasible path quickly. Common single-query algorithms include the Probabilistic Roadmap Method (PRM) and the Rapidly-exploring Random Tree (RRT).

2) **Asymptotically optimal algorithms**

   Asymptotically optimal algorithms gradually improve the path they have found until the optimal solution is reached; RRT* is an example of this category.

The advantages of sampling-based path planning algorithms are their wide applicability, their simple parameters and their good planning performance in high-dimensional spaces. They do not require explicit modelling of the state space; instead, random sampling and collision checking are used to explore the state space and to generate feasible paths. Therefore, they can be applied not only to the motion planning of robotic manipulators, but also to path planning problems for robots, unmanned aerial vehicles and other fields.

However, sampling-based path planning algorithms also have some disadvantages. First, randomly selecting nodes during expansion produces a number of useless nodes, which reduces the efficiency of the algorithm. Second, the algorithm may encounter local optima during the search, so that the optimal path cannot be found. In addition, for environments containing narrow passages the algorithm may fail to find the optimal path, because the chance of expanding nodes in these regions is relatively small.

2.1.2 The RRT Algorithm

Among the sampling-based path planning algorithms, one of the best known is the Rapidly-exploring Random Tree (RRT). RRT is a single-query path planning algorithm that aims to find a feasible path from the start point to the goal point as quickly as possible. Its core idea is to explore the search space by random sampling and the expansion of a tree structure, so as to discover the optimal path.

The basic implementation of the RRT algorithm is as follows:

- **Initialise the tree structure**: take the start point as the root node x_{init} and add it to the node set of the random tree.

- **Random sampling**: randomly pick a node x_{rand} from the feasible region as the target node. In practice, the goal point is often sampled directly as the target node with a certain probability in order to speed up the search.
- **Find the nearest node**: for the target node, find the node in the generated tree that is closest to x_{rand} and take it as the nearest node x_{near}.
- **Node expansion**: extend from the nearest node x_{near} along the direction of the line connecting it to x_{rand} by a fixed step length u to generate a new node x_{new}. If an obstacle lies on this direction, the expansion in that direction is abandoned.
- **Path checking**: check whether the newly generated node x_{new} lies inside the feasible region and does not intersect any obstacle. If the conditions are satisfied, the new node x_{new} is added to the tree structure and the edge between its parent node x_{near} and the new node x_{new} is added to the edge set of the tree.
- **Goal checking**: check whether the newly generated node is close to the goal point. If the new node is close enough to the goal, an attempt is made to connect it directly to the goal and to generate the final path.
- **Iterative search**: repeat the steps above until the termination condition is met. Usually a maximum number of iterations or a search time limit is set.
- **Path generation**: using the tree structure obtained by the search, backtrack from the goal point to the start point to generate the final path.

The node expansion process is illustrated in Figure 2.1:

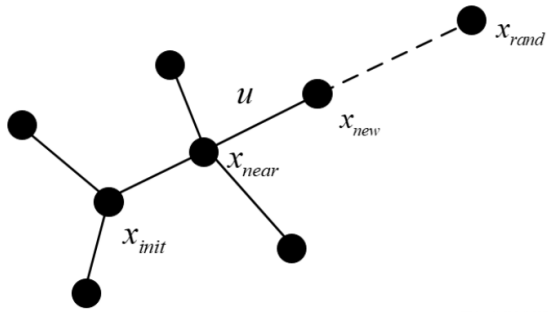


Figure 2.1: Flow chart of the RRT algorithm

The advantages of the RRT algorithm are its wide applicability, its simple parameters and its excellent planning performance in high-dimensional spaces. It does not require explicit modelling of the free region of the state space; instead, the feasibility of a path is verified through collision checking, so it can be applied to path planning problems in high-dimensional spaces such as robots and unmanned aerial vehicles.

2.2 Implementation and Optimisation of the Algorithm

In the code given in the appendix (Listing 7) the concrete implementation of the RRT algorithm can be seen. In the following, the optimised parts of the code are briefly explained. These optimisations mainly concern the code itself and are not illustrated with figures here; the optimised results and comparisons are given in the following section.

2.2.1 Collision-Checking Algorithm

Following the question raised in class, two cases were considered in the collision algorithm:

1) First, the distance from either endpoint of the generated line segment to the centre of the circle is smaller than the radius; this can be used as a criterion for a collision;

2) The case in which the distance from the line on which the segment lies to the centre of the circle is smaller than the radius:

   * if the angle between the endpoint and the centre of the circle is smaller than 90°, no collision has occurred;
   * if the angle between the endpoint and the centre of the circle is larger than 90°, a collision has occurred.

In this way the completeness of the collision algorithm is improved.

2.2.2 Safety-Radius Optimisation

When designing the collision-avoidance algorithm, the radius of the unmanned ship was added to the algorithm as a parameter. If the size of the unmanned ship is not taken into account, the motion path may visibly become almost tangent to an obstacle, which may cause the unmanned ship to collide during navigation. Taking the radius of the unmanned ship into account in the algorithm therefore significantly reduces the probability of a collision.

2.2.3 Turning-Angle Constraint Optimisation

When extending the parent node, the turning angle is restricted to no more than 20°, which avoids excessively large turns in the path. The corresponding code is given in Listing 1.

Code Listing 1: **Excerpt of the turning-angle constraint**

```
    def steer(self, from_node, to_node, extend_length= float("inf")):
        d, theta = self.calc_distance_and_angle(from_node, to_node)
        max_angle_diff = math.radians(20)
        if from_node.parent is not None:
            parent_angle = math.atan2(from_node.y - from_node.parent.y, from_node.x - from_node.parent.x)
            angle_diff =  abs(theta - parent_angle)
            if angle_diff > max_angle_diff:
                theta = parent_angle + max_angle_diff if theta > parent_angle else parent_angle - max_angle_diff
```

2.2.4 Probability-Concentration Optimisation

Since the RRT algorithm scatters points randomly, a certain amount of computing resource may be wasted, the path planning may become slow, and U-turns or wide arcs may appear. The algorithm was therefore improved so that a random point now has the highest probability of appearing in the direction of the goal point and the lowest probability of appearing in the opposite direction, which effectively reduces large bifurcations at the beginning of the path.

2.2.5 Path-Pruning Optimisation

The start point data and the obstacle list exported from demo3 were converted in coordinates: the longitude and latitude coordinates were transformed into Cartesian coordinates, and the

RRT model was then run on this data, with the start point at [0.0, 163.32] and the goal point at [195.45, 0.0].

It was found during this process that when the overall map becomes larger, the small step length means that the path is not smooth enough and a sawtooth shape appears. The finally generated path line was therefore pruned, which shortened the total distance and the total turning angle.

Through the longitude–latitude conversion, a 167×243 grid map with an approximate unit of 1 m was obtained, on which the RRT algorithm was implemented, and the finally generated path was pruned. The code used is given in Listing 2.

Code Listing 2: **Excerpt of the path-pruning routine**

```
    def smooth_path(self, path):
        pruned_path = [path[0]]
        i = 0
        while i <  len(path) - 1:
            # Look for the next pruning point
            j = i + 1
            while j <  len(path) and self.line_collision_free(pruned_path[-1], path[j], self.obstacle_list):
                j += 1
            pruned_path.append(path[j-1])
            i += 1

        # Append the goal point to the smoothed path
        pruned_path.append(path[-1])
        unique_path = [pruned_path[-1]]
        for point in  reversed(pruned_path[:-1]):
            if point not in unique_path:
                unique_path.insert(0, point)

        return unique_path
```

2.2.6 Evaluation-Criterion (Multiple-Iteration) Optimisation

The simplest path length was used as the evaluation criterion, and the main function was iterated 100 times: the path length generated by each planning run was calculated and the shortest one was selected as the final path. This effectively reduces the random part of the RRT algorithm and at the same time yields a simpler and smoother curve as the navigation path of the unmanned ship.

The code used for this part is given in Listing 3.

Code Listing 3: **Excerpt of the evaluation-and-iteration routine**

```

def run_rrt():
    # Initialise the map
    start_x, start_y, goal_x, goal_y, obstacle_list = initialize_map()
    start_time = time.time()  # Record the start time
    shortest_path = None
    shortest_length =  float('inf')
    path_lengths = []
    path_turnings = []
    path_numbers = []
    range_number = 200
    sum_number = 0

    # Carry out the path planning
    for _ in  range(range_number):
        rrt = RRT(
            start=[start_x, start_y],
            goal=[goal_x, goal_y],
            rand_area=[0, 250],
            obstacle_list=obstacle_list,
            play_area=[0, 300, 0, 200],
```

```
            robot_radius=4.0
        )

        valid_path = rrt.planning(animation=False, camera=None)

        # If a valid path has been found, update the shortest path
        if valid_path is not None:
            sum_number += 1
            final_path_nodes = [rrt.Node(x, y) for (x, y) in valid_path]
            smoothed_path = rrt.smooth_path(final_path_nodes)
            path_length = calculate_path_length(smoothed_path)
            path_lengths.append(path_length)
            path_turning = calculate_path_turning(smoothed_path) * 180 / math.pi
            path_turnings.append(path_turning)
            path_numbers.append( len(smoothed_path))
            if path_length < shortest_length:
                shortest_length = path_length
                shortest_path = smoothed_path


    # If the shortest path has been found, visualise and output it
    if shortest_path is not None:
        visualize_result(rrt, shortest_path)
    end_time = time.time()  # Record the end time
    total_time = end_time - start_time  # Compute the total time

    # Write the data to an Excel sheet
    data = {
        'Path Length': path_lengths,
        'Path Turning': path_turnings,
        'Path Number': path_numbers,
    }
    df = pd.DataFrame(data)
    df.to_excel('path_data.xlsx', index=False)
```

2.2.7 Code-Structure Optimisation

Object-oriented programming (OOP) is an important programming paradigm: by encapsulating data and behaviour in objects it improves the modularity, reusability and maintainability of the code. In the code, the RRT algorithm is implemented as a class, and concepts such as nodes and area bounds are abstracted into classes as well, which makes the code structure clearer and easier to understand and maintain. See Listing 4.[7]

It can be clearly seen in the code that most of the functions are encapsulated in the RRT class, and the `Main` function contains only a single line of code, which provides a good basis for the subsequent assembly of the code.

Code Listing 4: **Encapsulation of the RRT class**

```
class RRT:
    class Node:
        def __init__(self, x, y):

    class AreaBounds:
        def __init__(self, area):

    def __init__(self, start, goal, obstacle_list, rand_area, expand_dis=10.0, goal_sample_rate=5, max_iter=1000,
                 play_area=None, robot_radius=0.0):

    def planning(self, animation=True, camera=None):

    def steer(self, from_node, to_node, extend_length= float("inf")):

    def generate_final_course(self, goal_ind):

    def smooth_path(self, path):

    @staticmethod
```

```
    def line_collision_free(node1, node2, obstacle_list):

    def calc_dist_to_goal(self, x, y):

    def sample_free(self):

    def draw_graph(self, rnd=None, camera=None):

    @staticmethod
    def plot_circle(x, y, size, color="-b"):

    @staticmethod
    def get_nearest_node_index(node_list, rnd_node):

    @staticmethod
    def is_inside_play_area(node, play_area):

    @staticmethod
    def obstacle_free(node, obstacle_list, robot_radius):

    @staticmethod
    def calc_distance_and_angle(from_node, to_node):

def run_rrt():

def main():
    run_rrt()

if __name__ == '__main__':
    main()
```

2.3 Introduction to the LOS Guidance Algorithm

The navigation principle of the line-of-sight (LOS) method is reflected in its intuitive understanding of helmsman steering and ship motion. The principle holds that if the heading of the controlled ship is kept aligned with the line-of-sight angle (LOS angle), then, under appropriate control, the controlled ship will be able to reach the expected position and thus achieve path tracking. Moreover, the LOS algorithm reduces the conventional control variables from the ship position and heading angle of three degrees of freedom to the heading angle and the sailing speed of two degrees of freedom, and this property is particularly important for the control of underactuated ships.

2.3.1 Obtaining the Heading Angle

Since LOS theory holds that the controlled ship will reach the expected position as long as its heading is kept aligned with the LOS angle, how to obtain the LOS angle has become the focus of research.

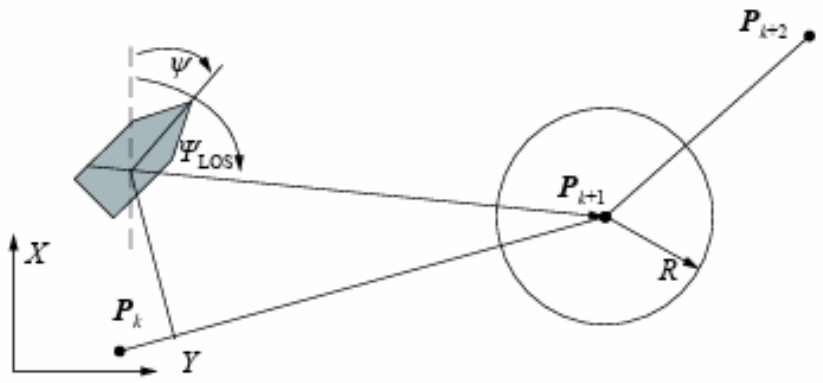


Figure 2.2: Principle of the LOS path-tracking control algorithm

As shown in Figure 2.2, assume that the current expected position for path tracking is $\boldsymbol{P}_{k+1} = [x_{k+1}, y_{k+1}]^T$ and that the previous expected position is $\boldsymbol{P}_k = [x_k, y_k]^T$; let the current position of the controlled ship be $\boldsymbol{P} = [x, y]^T$, and assume that $\boldsymbol{P}_{k+1}$ and $\boldsymbol{P}_k$ can be measured. The LOS angle can then be calculated from Equation 2.1:

$$\psi_{\mathrm{LOS}} = \arctan\left(\frac{y_{k+1} - y}{x_{k+1} - x}\right) \tag{2.1}$$

where $\psi_{\mathrm{LOS}} \in \langle 0, 2\pi \rangle$ is the LOS angle.

Because the control principle of the LOS method is easy to understand, its control effect is better than that of other control methods and its applicability is strong, the LOS method has been widely used in ship path-tracking control. The two navigation modes of the LOS method are analysed separately below.

2.3.2 Analysis of the LOS Navigation Modes

The LOS control algorithm can be used for both straight-line path tracking and curved-path tracking, and it is a good path-tracking control algorithm. The algorithm is assisted by the introduction of a line-of-sight distance; the principle is shown in Figure 2.3.

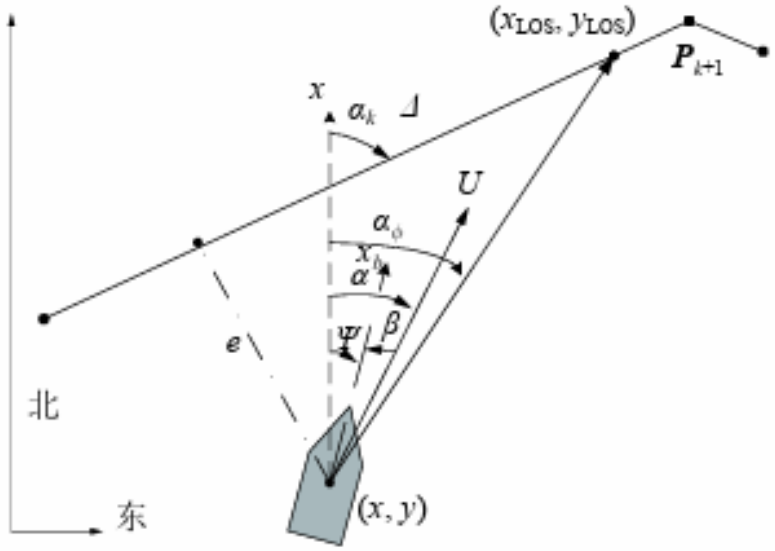


Figure 2.3: Illustration of the LOS path-tracking navigation mode

As can be seen from Figure 2.3, the expected heading point $\boldsymbol{P}_{\mathrm{LOS}} = [x_{\mathrm{LOS}}, y_{\mathrm{LOS}}]^{\mathrm{T}}$ is at a distance $\Delta = nL_{pp}$ from the projection point of the current position of the controlled ship on the expected track, where $n = 2 \sim 5$, L_{pp} is the ship length and Δ is called the line-of-sight distance of the controlled ship[8]. The LOS control algorithm can then be written as:

$$\alpha_{\phi} = \alpha_k + \arctan\left(\frac{-e}{\Delta}\right) \tag{2.2}$$

where e denotes the lateral following error of the controlled ship and α_k is the angle between due north of the geodetic coordinate system and the expected track line $\boldsymbol{P}_K$, $\boldsymbol{P}_{k+1}$. In this paper the heading-angle deviation is assumed to be $\psi_e = \alpha_{\phi} - \psi$; when ψ_e tends to zero, the self-propelled model sails towards the target heading point and finally reaches the expected track, so that path tracking is accomplished. Finally, by means of a simple PID control, ψ_e can be effectively driven towards zero, the current heading angle ψ then tracks the expected heading angle α_{ψ}, and through continuous control the self-propelled model can sail along the expected track.

After the ship has tracked the current expected heading point $\boldsymbol{P}_{k+1} = [x_{k+1}, y_{k+1}]^{\mathrm{T}}$ by means of the control algorithm, once the ship enters a certain range around this expected heading point, the tracking of this point must be abandoned automatically and switched to the tracking of the next expected heading point, that is, $k = k + 1$. A control algorithm is therefore needed to discriminate between and switch the expected heading points; the algorithm adopted in this paper is as follows.

Assume that there is a circle whose centre is the current expected heading point (x_{k+1}, y_{k+1}) tracked by the ship and whose radius is R_0. If at a certain instant the current position of the controlled ship $\boldsymbol{P} = [x, y]^T$ satisfies

$$(x_{k+1} - x)^2 + (y_{k+1} - y)^2 \leqslant R_0^2 \tag{2.3}$$

then the current expected heading point being tracked has to be changed to the next expected heading point. When ψ_{LOS} is calculated, the calculation must also be switched to the next expected heading point, and the data of that point are used for the calculation.

2.3.3 Design of the LOS-Based PID Controller

In order to make the heading-angle deviation obtained from the LOS control algorithm approach zero, that is, in order to make the current heading angle of the self-propelled model track the expected heading angle, a control method has to be adopted in this paper so that the deviation approaches zero and the purpose of path tracking is achieved. The PID control diagram is shown in Figure 2.4.

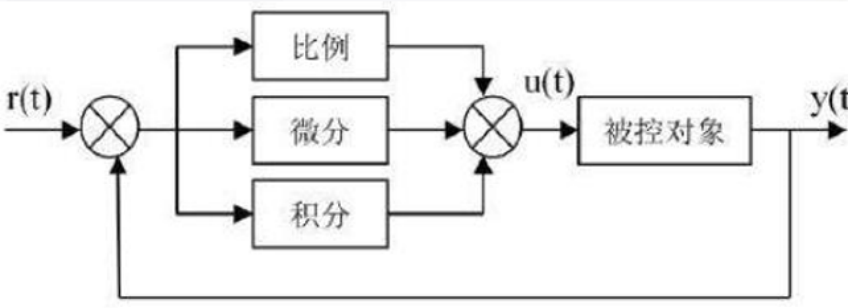


Figure 2.4: Schematic diagram of the PID control

It can be seen from Figure 2.4 that a stable and convergent control law, which drives ψ_e towards zero, can be obtained by setting reasonable parameters of the PID controller. The control law of the PID controller is given in Equation 2.4:

$$\delta = P * (\Psi_e - \Psi_c) + I * \sum_i (\Psi_e - \Psi_c)_i + D * (\Psi_{ci} - \Psi_{c(i-1)}) \tag{2.4}$$

Since the rudder angle changes in proportion to the current heading angle, the heading-angle deviation is

$$\psi_e = \psi - \psi_{\text{LOS}}$$

It can be seen from the equations above that if at a certain instant the system has a relatively large ψ_e, the PID control will produce a rudder angle that steers the self-propelled model towards the expected heading point; the current heading angle of the model then approaches its expected heading angle and ψ_e decreases. Through continuous control, as long as ψ_e exists, a control rudder angle will be produced that reduces ψ_e until ψ_e tends to zero, that is, until the model has reached the expected track line.

2.4 Optimisation of the Path-Tracking Code

2.4.1 The PID Function

1) **Setting of K_p:**

The calculated error equals the target heading angle minus the current yaw angle and ranges over $(0, 360]$, while it is mapped onto `steer` over the range $(-1, 1]$. Therefore the value that K_p must reach in order to map the maximum angle completely was calculated in advance:

$$\frac{2}{360} = 0.0056$$

However, it should be noted that when the angles differ most, the difference between the heading angle and the yaw angle is 180, which gives the lower bound of K_p:

$$\frac{2}{180} = 0.011$$

The value of K_p at the lowest speed (`throttle = 0.4`) was therefore first set to 0.02, so that a full rudder turn can be performed when the heading deviation is too large. More refined values of K_p were obtained later through simulator experiments, see Table 2.1 (the formula is approximate reference data).

Speed	K_p
Low speed, throttle = 0.5	0.09
Medium speed, throttle = 0.8	0.13
High speed, throttle = 1.0	0.15
Any speed	$0.2 \times \sqrt{V_{throttle}} - 0.05$

Table 2.1: K_p parameters as a function of the speed

The concrete implementation of the code is given in Listing 5.

2) **Setting of K_i:**

The K_i term represents the accumulated difference between the heading and the expected heading over time. This part is not corrected for the moment, that is, $K_i = 0$.

3) **Setting of K_d:**

The K_d term represents the influence of the rate of change of the heading angle on the heading correction; the heading angle is corrected by the difference between the previous and the next instant, and a suitable choice should avoid overshoot caused by a heading angle that changes too quickly. The lowest speed currently selected is 0.4, so this part is relatively small and is chosen as 0.002, which prevents the angle from changing too slowly to reach the target point.

Code Listing 5: **PID parameters and implementation of the LOS algorithm**

```
def PID_controller(current_yaw, target_yaw, throttle):
    global pre_error, sum_error
    yaw_difference = target_yaw - current_yaw

    if  abs(yaw_difference) > 180:
        if yaw_difference > 0:
            error = yaw_difference - 360
        else:
            error = yaw_difference + 360
    else:
        error = yaw_difference

    sum_error += error * 0.1
    d_error = error - pre_error
    Kp = 0.2 * math.sqrt(throttle) - 0.05
    Ki = 0
    Kd = 0 * math.sqrt(throttle)
    steer = Kp * error + Ki * sum_error + Kd * d_error

    steer =  max( min(steer, 1), -1)

    pre_error = error

    return steer
```

2.4.2 Correction of the PID Control Angle

In theory the LOS path-tracking control algorithm described above is very effective. However, because the control interval of this algorithm is $(0, 2\pi)$, it is discontinuous at the two nodes 0 and 2π.

If, for example, the current heading angle of the ship is $\psi = 1°$ and the heading angle continues to change clockwise, then the next moment it may well be $\psi = 359°$; the algorithm would then consider that the ship has performed a very fast rotation and would give unpredictable results.

The occurrence of this situation is very unfavourable for the path-tracking control of a ship: it makes the design of the controller difficult and may cause the controller to issue an erroneous command while the ship is tracking the expected heading point, so that the ship tracks that point in the opposite direction along a large arc. All of this has a great impact on the actual control of the ship.

It was therefore found that the range can be extended to $(0, 4\pi]$: in the part where the error is calculated, the absolute value of `target_yaw - current_yaw` is compared with the absolute value of `target_yaw + 2π - current_yaw`, and the smaller of the two is taken as the value of the error.

In this way, when the angle varies between approximately 0 and 2π, if the yaw angle takes a value in $(180, 360]$, the data close to 0 are discarded artificially and a value close to $0 + 360$ is used instead as the target heading angle. Values of the yaw angle in $(0, 180)$ are treated in a similar way, which avoids the influence of a sudden change of the heading angle at due north. The concrete implementation of the code is given above, in Listing 5.

2.4.3 Throttle Control and the Corresponding Adjustment of the PID Parameters

A fixed speed is quite restrictive: at high speed a large deviation from the route appears at the turning points, whereas at low speed the completion time becomes too long. We therefore wished to implement throttle control as an optimisation of the control algorithm. The concrete implementation is as follows.

At present the line-of-sight distance of the simulated ship is $\Delta = 5$ m, so we consider that the speed should be reduced as soon as the ship reaches the line-of-sight distance and starts to turn, which effectively reduces the deviation from the route; at the same time, the ship advances at full speed in places far from the turning points, which reduces the completion time and improves the rapidity of the ship.

Take Figure 2.5 as an example.

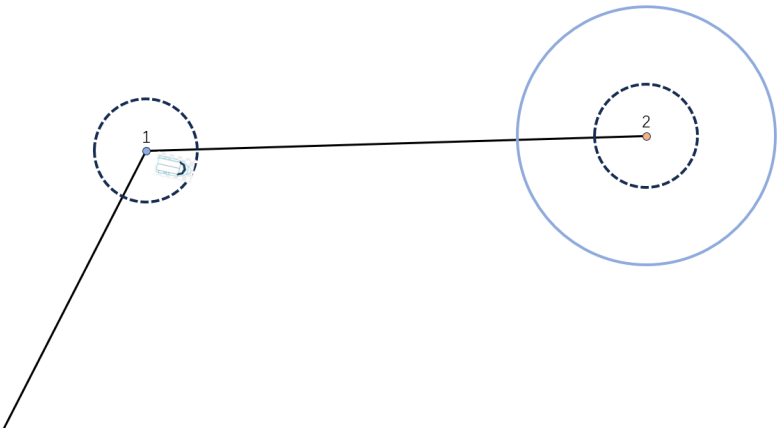


Figure 2.5: Schematic diagram of the throttle control

In the figure the test ship has already passed point 1 and the target point is point 2. The dashed small circle has a radius of 5 m and marks the range in which the throttle is set to 0.4; the blue large circle has a radius of 10 m and marks the range in which the throttle is set to 0.8; outside that range the throttle is set to 1.0. The purpose of this setting is to reduce the speed to 0.8 in advance when the ship is approaching the target point, so that the ship does not overshoot the adjustment range at too high a speed; at the same time, when the ship has reached the line-of-sight distance and begins to turn towards the next target point, the speed is reduced to 0.4 until the ship has passed the dashed range of the target point, which makes it possible to complete the turn along a smaller arc and reduces both the possibility of oscillation after passing the target point and the duration of such oscillation. The concrete implementation is given in Listing 6.

Code Listing 6: **Implementation of the throttle control optimisation**

```
1 def LOS(current_position, target_point, Delta = 5):
2
3     def calculate_distance(lat1, lon1, lat2, lon2):
```

```
    def calculate_angle(lat1, lon1, lat2, lon2):

    distance_to_target = calculate_distance(current_position['latitude'], current_position['longitude'],
                                            target_point['latitude'], target_point['longitude'])
    target_point_next = path[path_index - 1]
    distance_to_target_next = calculate_distance(current_position['latitude'], current_position['longitude'],
         target_point_next['latitude'], target_point_next['longitude'])
    if distance_to_target < 0.005:
        if path_index >=  len(path) -1:
            if distance_to_target < 0.0035:
                path_index += 1
                return 0, 0, current_position['latitude'], current_position['longitude']
        if path_index <  len(path) - 1:
            path_index += 1
            target_point = path[path_index]
            target_point_next = path[path_index - 1]

    if distance_to_target < 0.005 or distance_to_target_next < 0.005:
        throttle = 0.4
    if distance_to_target > 0.005 and distance_to_target < 0.02 and distance_to_target_next > 0.005:
        throttle = 0.8
    if distance_to_target > 0.02 and distance_to_target_next > 0.005:
        throttle = 1.0

    if target_point_next['latitude'] != target_point['latitude']:
    else:
    angle = calculate_angle(current_position['latitude'], current_position['longitude'],
                            DotLos['latitude'], DotLos['longitude'])
    return angle, throttle, x_perpendicular, y_perpendicular
```

3 Experimental Results and Analysis

3.1 Results and Optimisation of the Planning Algorithm

3.1.1 Evaluation Metrics

For the planning algorithm we consider that, provided a path is planned that does not collide with any obstacle, the more important indicators of the path result are the path length, the number of turning points and the path turning angle, while the processing time is a secondary indicator.

The path length, the sum of the turning angles and the processing time can therefore be used as evaluation metrics. Since the number of turning points, the sum of the turning angles, the length and the time are not expressed in the same unit, the data are first normalised, and the effect is then judged from the normalised data.

At the same time it has to be taken into account that the weights of the individual evaluation metrics should differ: in the planning stage the path length and the number of turning points should be the most important indicators, the sum of the turning angles comes second, and the processing time comes last. The analytic hierarchy process (AHP) was therefore used to calculate the weights. By comparing each pair of influencing factors, the following judgement matrix based on the importance ratings is obtained:

$$\begin{pmatrix} & \text{Path length} & \text{Turning-angle sum} & \text{Turning points} & \text{Processing time} \\ \text{Path length} & 1 & 2 & 3 & 5 \\ \text{Turning-angle sum} & \frac{1}{2} & 1 & 2 & 4 \\ \text{Turning points} & \frac{1}{3} & \frac{1}{2} & 1 & 3 \\ \text{Processing time} & \frac{1}{5} & \frac{1}{4} & \frac{1}{3} & 1 \end{pmatrix}$$

In order to check whether the matrix is inconsistent, a consistency test has to be carried out on the matrix above in order to decide whether it can be accepted.

The consistency test is based on the consistency ratio (CR), which is calculated as follows:

$$CR = \frac{CI}{RI}$$

where CI is the consistency index and RI is the random index. The consistency index is calculated as $CI = \frac{\lambda_{max}-n}{n-1}$, where λ_{max} is the largest eigenvalue of the judgement matrix. After calculation, $CI = 0.00703652$.

Finally, $CR = 0.00892946 < 0.1$ is obtained, so the consistency of the judgement matrix is considered acceptable and the judgement matrix is therefore regarded as valid.

The weights of the evaluation metrics of the planning algorithm obtained by the AHP method are given in Table 3.1.

Metric	Path length	Turning-angle sum	Turning points	Processing time
Weight	0.47	0.28	0.18	0.07

Table 3.1: Weights of the evaluation metrics of the planning algorithm

The following formula is therefore used to calculate the final score of a planned path in the evaluation of the optimisation results (all parameters in the calculation are normalised indices; the normalisation method is described below):

$$S_{RRT} = 0.47\hat{L} + 0.28\hat{T}_{sum} + 0.18\hat{D}_{sum} + 0.07\hat{t} \tag{3.1}$$

where S_{RRT} is the score finally obtained by the path, L is the path length, T_{sum} is the sum of the turning angles, D_{sum} is the number of turning points and t is the processing time of the program.

The normalisation is carried out as follows (since all four indicators are better when they are smaller, the maximum value minus the original value is used for the normalisation):

$$\text{Normalised value} = \frac{\text{Maximum value} - \text{Original value}}{\text{Maximum value} - \text{Minimum value}}$$

Note: if the normalised value is smaller than 0 because of the choice of the upper and lower limits, it is treated directly as 0.

The maximum and minimum values of each evaluation metric are chosen as follows:

1) **Path length:**

The direct connection of the start point and the goal point is taken as the shortest path length and the orthogonal polyline is taken as the longest path length for the normalisation of the planned path length, as shown in Figure 3.1.

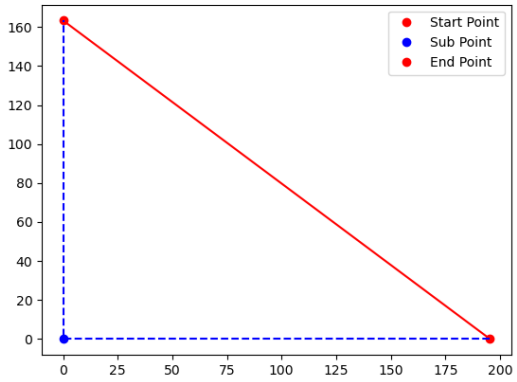


Figure 3.1: Illustration of the upper and lower limits of the path length

In the figure the red line is the shortest path, with a length of 254.711, and the blue dashed line is the defined longest path, with a length of 358.78.

2) **Sum of the turning angles:**

Since the sum of the turning angles has no intuitive representation, 0° is taken directly as the minimum value and 1500° as the maximum value for the normalisation.

3) **Number of turning points:**

After the initial RRT algorithm without any optimisation was run 1000 times, the average number of turning points was about 42, so the maximum number of turning points is taken as 42 and the minimum as 2 (the start point is included in the number of turning points).

4) **Processing time:**

According to the processing-time tests of the program before and after the optimisation, it was found that when the number of iterations in the evaluation-criterion optimisation is 200, the processing time is about 13.5 s. A maximum value of 15 s and a minimum value of 0 s are therefore chosen for the normalisation.

3.1.2 Visual Results of the Individual Optimisations

3.1.2.1 Optimisations Applied Separately The results obtained after applying the different optimisations separately are shown below.

1) **Turning-angle constraint alone:**

The image obtained after using the turning-angle constraint optimisation is shown in Figure 3.2.

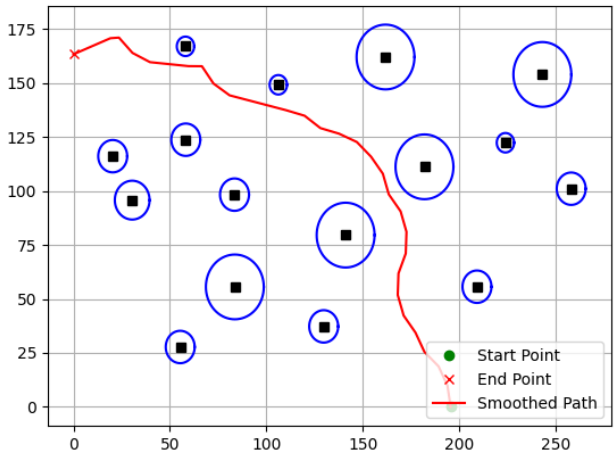


Figure 3.2: Path generated by the RRT algorithm after the turning-angle constraint optimisation

It can be seen from the resulting image that after the turning-angle constraint optimisation the path itself is relatively smooth, but the path length itself is not reduced by much and the path still makes a fairly large arc.

The detailed data analysis is given in the following section.

2) **Probability concentration alone:**

The image obtained after using the probability-concentration optimisation is shown in Figure 3.3.

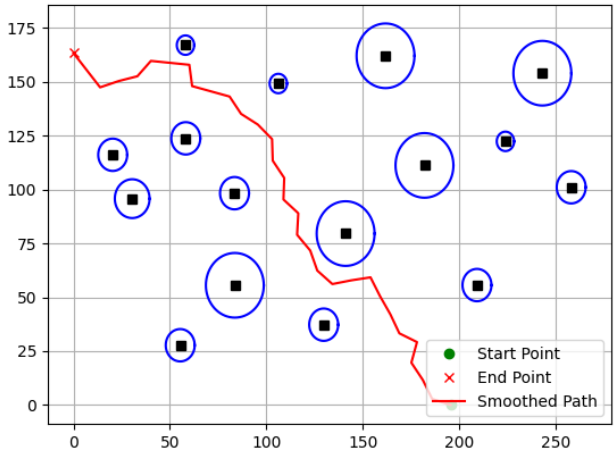


Figure 3.3: Path generated by the RRT algorithm after the probability-concentration optimisation

It can be seen from the resulting image that after the probability-concentration optimisation the generated path is more inclined towards the direction of the goal point and the path length is reduced accordingly, but the turning angles are not improved.

3) **Path pruning alone:**

The image obtained after using the path-pruning optimisation is shown in Figure 3.4.

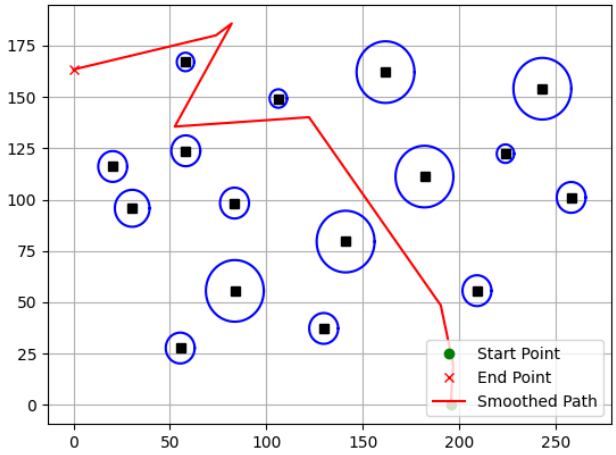


Figure 3.4: Path generated by the RRT algorithm after the path-pruning optimisation

It can be seen from the resulting image that after the path-pruning optimisation the number of turning points of the path is greatly reduced, so the total sum of the turning angles decreases, but the number of turning points with a large turning angle increases; the path length is also reduced accordingly, but the overall path is still rather long.

4) **Multiple iterations alone:**

The image obtained after using the multiple-iteration optimisation is shown in Figure 3.5.

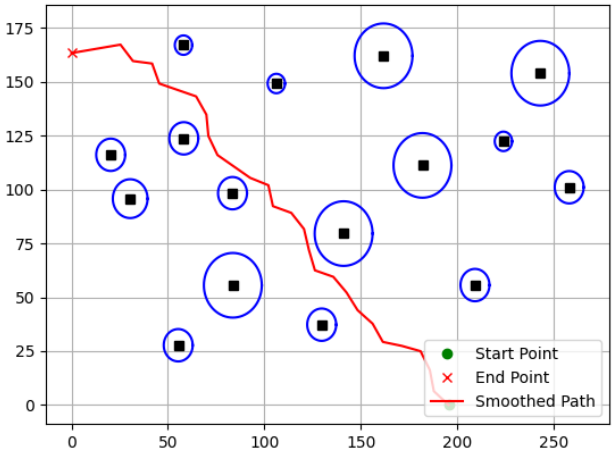


Figure 3.5: Path generated by the RRT algorithm after the multiple-iteration optimisation

It can be seen from the resulting image that after the multiple-iteration optimisation, since the evaluation metric used in the iterations is the path length, the total path length is greatly reduced, but the sum of the turning angles is still not improved much and the overall path is rather distorted.

3.1.2.2 Optimisations Applied in Combination Different optimisation methods were then combined with each other, which gave the following result images:

1) **Probability + turning-angle optimisation:**

After using the probability-concentration and the turning-angle optimisation at the same time, the image shown in Figure 3.6 is obtained.

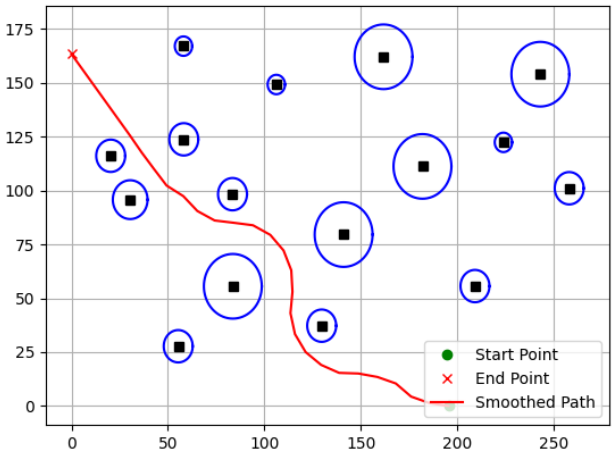


Figure 3.6: Path generated by the RRT algorithm after the probability + turning-angle optimisation

It can be seen from the image that when the probability-concentration optimisation and the turning-angle constraint optimisation are used at the same time, the turning angle of each turning point is kept from becoming too large while the probability of generating the path towards the goal point is higher.

The resulting path is shorter and at the same time has a smaller sum of turning angles, so on the whole the effect is good.

2) **Turning-angle + pruning optimisation:**

After using the turning-angle constraint and the path-pruning optimisation at the same time, the image shown in Figure 3.7 is obtained.

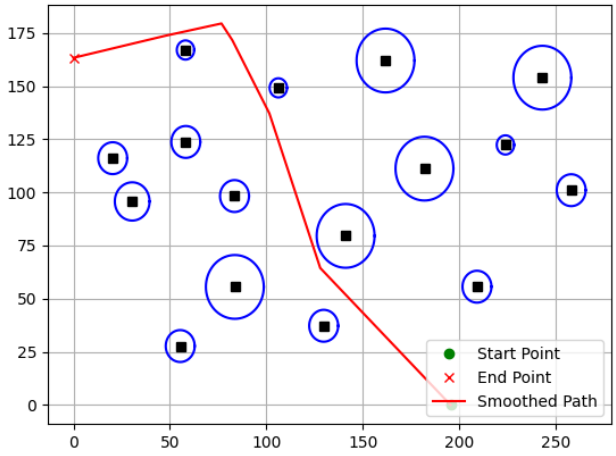


Figure 3.7: Path generated by the RRT algorithm after the turning-angle + pruning optimisation

It can be seen from the image that after the turning-angle constraint and the path pruning are used, although the initial direction deviates slightly, which produces a fairly large polyline at the beginning of the path, the subsequent processing is rather good: in the middle and later part of the path the length is compressed while the sum of the turning angles is also small.

3) **Iteration + pruning optimisation:**

After using the multiple iterations and the path pruning at the same time, the image shown in Figure 3.8 is obtained.

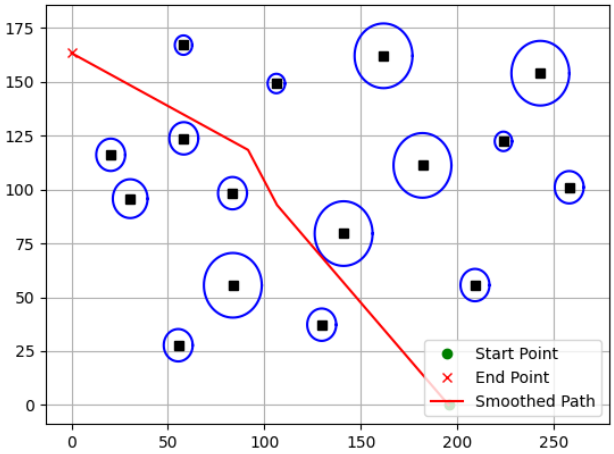


Figure 3.8: Path generated by the RRT algorithm after the evaluation + pruning optimisation

It can be seen from the image that after using the pruning and the multiple iterations, the resulting path is already very concise and meets the length requirement; compared with the most ideal case, only the number of turning points may be slightly larger, so that the perfect case of a route consisting of only two segments is not reached.

The reason may be that 100 iterations are somewhat too many, so that a fairly perfect curve can already be obtained by using pruning and multiple iterations alone. A further comparison between the combined method and the evaluation + pruning method will be carried out in the following data processing.

3.1.2.3 Combined Optimisation Finally, the result of the RRT algorithm after combining all the optimisation methods is shown in Figure 3.9.

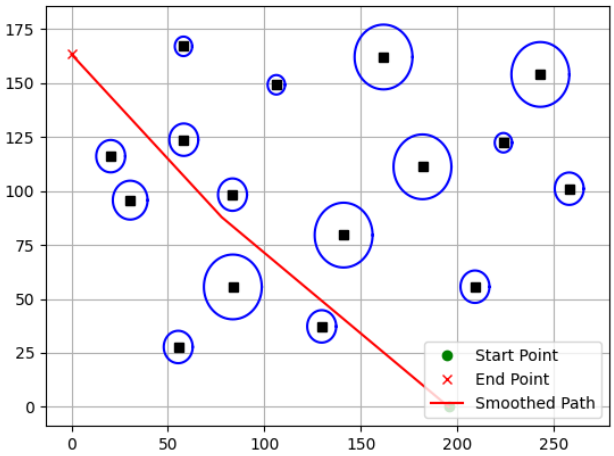


Figure 3.9: Path generated by the RRT algorithm after the combined optimisation

It can be seen that after the combined optimisation, that is, after combining the turning-angle optimisation and the probability concentration with the path pruning and after carrying out 100

iterations based on the evaluation metric, the resulting path has almost become perfect: there are only three turning points, and the path length, the turning angle and the other indicators are not very different from a direct connection of the start point and the goal point. A near-optimal solution has been reached.

3.1.3 Change of the Evaluation Metrics after Each Optimisation

After the intuitive analysis of the path images obtained with the individual optimisation methods, the data analysis is carried out below. All the data used below are the average values of 1000 program runs (for the program with multiple iterations, which has already iterated 100 times, only the average of 20 runs was taken for reasons of computing time).

3.1.3.1 Optimisations Applied Separately According to Equation 3.1, the overall score of the path-planning program after each single optimisation can be calculated, as shown in Figure 3.10; the concrete data are given in the table.

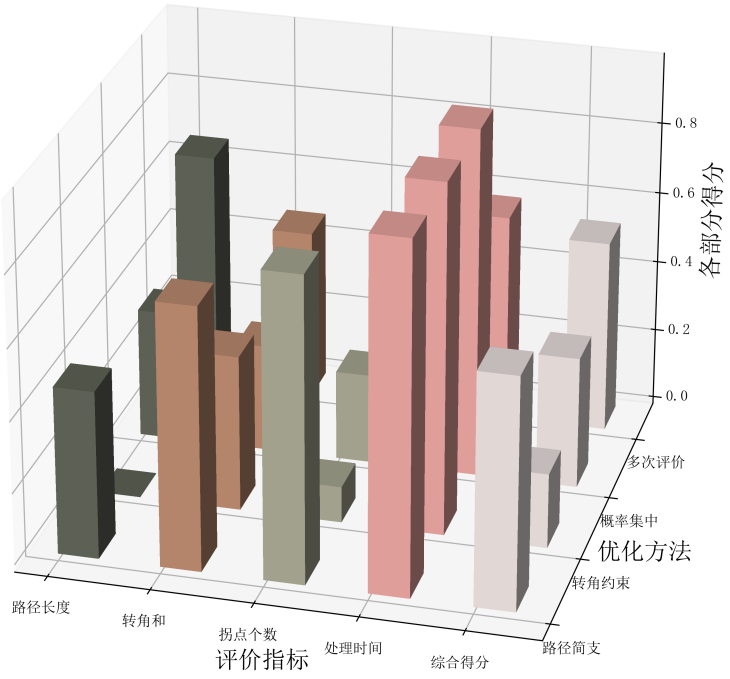


Figure 3.10: Scores of the path-planning program after single optimisations

Metric	Path pruning	Turning-angle constraint	Probability concentration	Multiple iterations
Path length	0.467	0.000	0.363	0.663
Turning-angle sum	0.733	0.434	0.300	0.474
Turning points	0.589	0.096	0.236	0.289
Processing time	0.977	0.977	0.976	0.582
Overall score	0.599	0.207	0.365	0.537

Table 3.2: Results of the path-planning program after single optimisations

It can be seen that with single optimisations the overall scores are not very high. Relatively speaking, the best performance is achieved by the program with path pruning; the reason is presumably that the number of nodes is small, so that the score for the number of turning points is particularly high.

The second best performance is achieved by the algorithm with the multiple-iteration optimisation. This may be because the iterations are based on the path length, so that the score of the path-length indicator, which has the largest weight, is relatively high; this is in fact the case.

3.1.3.2 Optimisations Applied in Combination In the same way, the overall score of the path-planning program after the combined optimisation is calculated according to Equation 3.1, as shown in Figure 3.11; the concrete data are given in Table 3.3.

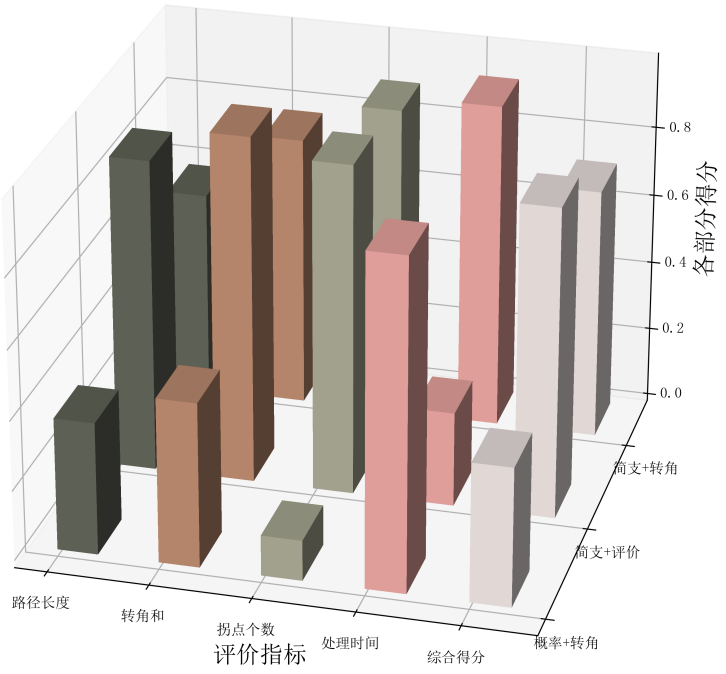


Figure 3.11: Scores of the path-planning program after combined optimisations

Metric	Probability + turning angle	Pruning + evaluation	Pruning + turning angle
Path length	0.378	0.896	0.585
Turning-angle sum	0.469	0.994	0.779
Turning points	0.115	0.946	0.896
Processing time	0.944	0.270	0.936
Overall score	0.396	0.889	0.720

Table 3.3: Results of the path-planning program after combined optimisations

It can be seen that as soon as the evaluation-based multiple-iteration optimisation is added, the score of the time indicator becomes relatively low while the path length gains a great deal; at the same time, after the pruning is added, both the number of turning points and the sum

of the turning angles gain to a certain extent, whereas the other two optimisations have only a moderate effect. This result is consistent with the analysis of the single optimisations.

At present, if there is a limit on the number of optimisations, the best method is to use the pruning together with the multiple iterations, which is the best solution so far. In the following, however, the combined optimisation method that uses all the optimisation methods together, together with its result data, is presented, and the necessity of adding the probability concentration and the turning-angle optimisation is explained.

3.1.3.3 Combined Optimisation In the combined optimisation, the best path out of 100 iterations was selected according to the path-length criterion, and this was taken as one simulation of the algorithm. The average data of 27 simulations gave the score parameters shown in Table 3.4.

	Path length	Turning-angle sum	Turning points	Processing time	Overall score
Raw data	255.222	7.336	3.156	12.531	
Normalised score	0.995	0.995	0.971	0.165	0.933

Table 3.4: Results of the path-planning program after the combined optimisation

In the table, the units of the raw data are, in order: metre, degree (°), number of points and second.

It can be seen that in terms of the path length, the sum of the turning angles and the number of turning points, the results of the combined optimisation are basically full marks: the path length is essentially equal to the connecting straight line, the sum of the turning angles tends to 0, and the number of turning points is basically kept between 3 and 4 and is close to 3. The overall score is the highest; only the processing time is somewhat low, so that the score does not come close to full marks.

The overall scores of the individual optimisations are compared with that of the combined optimisation in Figure 3.12.

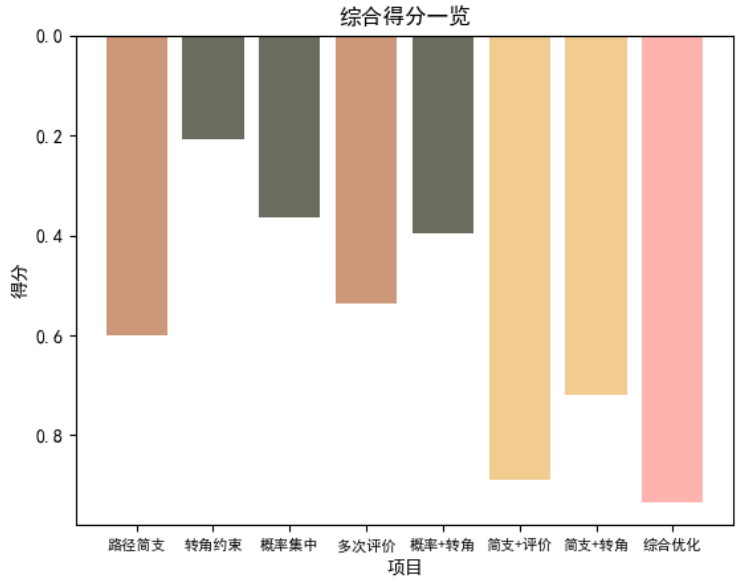


Figure 3.12: Comparison of the overall scores

It can be seen that the pruning and the multiple iterations improve the result of the path-planning algorithm greatly, and that the effect of the combined optimisation is indeed the best.

The necessity of adding the turning-angle constraint and the probability concentration to the combined optimisation is discussed below. Since the number of iterations is currently quite large, the score of the pruning + iteration optimisation alone is not very different from that of the combined optimisation, and the processing time of the combined optimisation is somewhat longer; the question therefore arises whether deleting the turning-angle constraint and the probability concentration would affect the effect of the combined optimisation to any great extent. The individual iterations are therefore separated below, and the convergence curves of the path-planning algorithm for the combined optimisation and for the pruning + iteration method are discussed.

The evaluation criterion is the overall score obtained from the three items path length, sum of the turning angles and number of turning points, without taking the time into account. The formula is as follows.

$$S_{RRT} = 0.47\hat{L} + 0.28\hat{T}_{sum} + 0.18\hat{D}_{sum}$$

After each iteration, the currently best planned path of the algorithm was selected and its overall score was calculated, and the way in which the overall score varies with the number of iterations was observed. The total number of iterations was 10, and the data are the average values of 10 tests of the model.

The results of the planning are shown in the figure below.

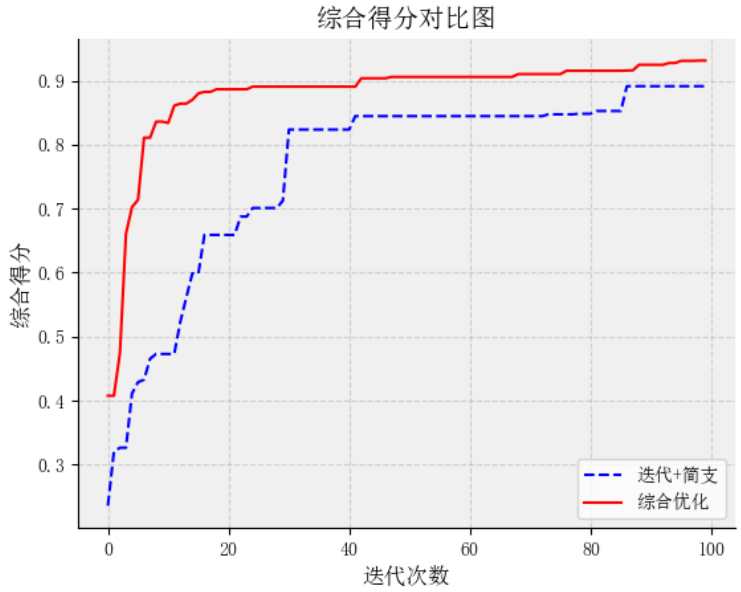


Figure 3.13: Convergence of the overall score

It can be seen from the figure that the combined optimisation converges much faster than the calculation with pruning + iteration alone, but after 100 iterations the data tend to the same values and both have approximately reached a perfect path.

It can therefore be considered that the combined optimisation gives slightly better results than the iteration + pruning method while saving a considerable amount of computing resource, that is, the probability concentration and the turning-angle constraint are worth adding and are of some help in optimising the path-planning algorithm.

The combined optimisation was therefore finally chosen as the final algorithm for path planning; the concrete code is given in Listing 7.

3.2 Results and Optimisation of the Control Algorithm

3.2.1 Evaluation Metrics

As with the planning algorithm, we consider that for the control algorithm the sailing speed of the simulated ship is the most important indicator, which is represented by the completion time; the indicator that represents the manoeuvrability is also quite important and can be expressed by the difference between the accumulated turning angle of the hull during the simulated voyage and the sum of the turning angles of the planned path; measuring whether the test ship deviates from the route is also a fairly important indicator and can be represented by the accumulated distance by which the hull deviates from the route; finally there is the length of the whole route, which can be used to calculate economic parameters such as the fuel consumption.

In the same way, the AHP method is used to calculate the weights of the individual indicators from the normalised data in order to obtain an overall score. After similar steps, the following weight matrix and parameters were obtained:

	Completion time	Turning-angle difference	Deviation error	Path length
Completion time	1	2	3	3
Turning-angle difference	$\frac{1}{2}$	1	2	2
Deviation error	$\frac{1}{3}$	$\frac{1}{2}$	1	1
Path length	$\frac{1}{3}$	$\frac{1}{2}$	1	1

The consistency indices obtained by the program are as follows:

$$CI : 0.0034543 \quad CR : 0.0038381$$

The consistency test is satisfied, and the weight parameters given in Table 3.5 are obtained.

Metric	Completion time	Turning-angle difference	Deviation error	Path length
Weight	0.457	0.263	0.140	0.140

Table 3.5: Weights of the evaluation metrics of the control algorithm

The normalisation step follows.

Since there is no fairly standard measure in the control algorithm, the maximum and minimum values used for the normalisation are taken from the test results; the results of the individual optimisation processes are therefore obtained and presented first, and the overall scores are calculated and compared only at the end. The concrete formula is similar to the previous one.

$$\text{Normalised value} = \frac{\text{Maximum value} - \text{Original value}}{\text{Maximum value} - \text{Minimum value}}$$

3.2.2 Change of the Evaluation Metrics after Each Optimisation

Among the optimisations, the logic of the control algorithm was not changed to any great extent; only the throttle control and the heading-angle mapping were added. Without the latter the simulated ship cannot run as far as the goal point. In the following, therefore, only the parameters Δ, K_p, K_d and the throttle value of the control algorithm without the throttle regulation are adjusted, and the results are compared with those of the algorithm to which the throttle control has been added.

In the subsequent tests of the algorithm results, the shortest planned path is too simple to be of much interest for the adjustment, so the rather curved path with more turning points shown in Figure 3.14 was chosen as the test path for the control algorithm; the parameters of this path are given in Table 3.6.

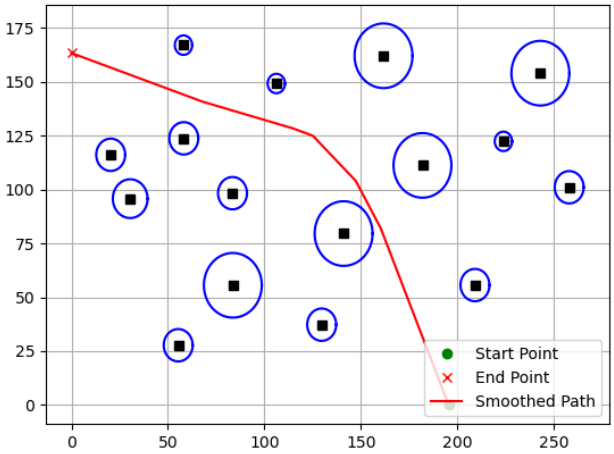


Figure 3.14: Test path of the planning algorithm

Metric	Path length	Total turning-angle sum
Value	276.209	55.473

Table 3.6: Parameters of the test path of the planning algorithm

3.2.2.1 Results of the Control Algorithm without Throttle Control The ship speed is divided into three levels: low speed, throttle = 0.5; medium speed, throttle = 0.8; and high speed, throttle = 1.0. At the same time, four levels of the control parameters are used for each ship speed (since the influence of K_d is relatively small, it was not adjusted this time); the levels of the control parameters differ between the ship speeds. Four groups of experiments were carried out in total and their average values were analysed; the concrete parameters are given in Table 3.7. From these motion evaluation parameters the upper and lower limits of the normalisation can be obtained; the values in the vicinity of the maximum and minimum values in Table 3.7 that are fairly regular and easy to calculate are taken as the limits, and the concrete data are given in Table 3.8.

Metric	Low speed				Medium speed				High speed			
K_p	0.03	0.05	0.08	0.1	0.05	0.08	0.1	0.15	0.08	0.1	0.15	0.2
Sailing length	277.561	278.300	278.060	277.951	288.794	285.114	282.752	283.124	291.201	293.256	288.890	292.730
Turning-angle sum	287.600	262.401	230.901	286.601	619.000	511.800	474.800	460.200	836.584	639.289	622.024	909.079
Mean deviation	0.203	0.226	0.173	0.192	0.796	0.655	0.481	0.434	1.103	1.039	0.883	0.931
Completion time	228.970	239.243	232.662	232.217	152.486	174.534	158.977	156.838	130.552	140.297	133.309	143.500
Turning-angle difference	232.127	206.928	175.428	231.128	563.527	456.327	419.327	404.727	781.111	583.816	566.551	853.606
Length difference	1.352	2.091	1.851	1.742	12.585	8.905	6.543	6.915	14.992	17.047	12.681	16.521

Table 3.7: Evaluation metrics of the simulated runs of the control algorithm without throttle regulation

Metric	Maximum value	Minimum value
Average deviation	1.25	0.15
Completion time	250	125
Total turning-angle difference	900	150
Total length difference	20	0

Table 3.8: Normalisation parameters

After the normalisation had been completed, the control algorithm was scored for a series of different parameters. An example of low-speed operation is shown in Figure 3.15, an example of medium-speed operation in Figure 3.16 and an example of high-speed operation in Figure 3.17. A more concise data table was also obtained, see Table 3.9.

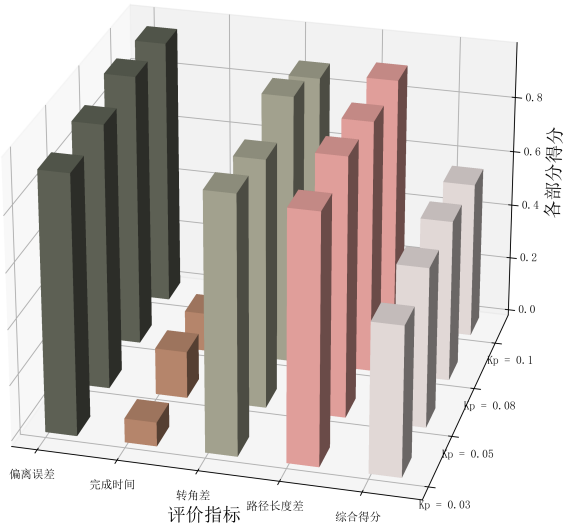


Figure 3.15: Comparison of the overall scores for low-speed operation

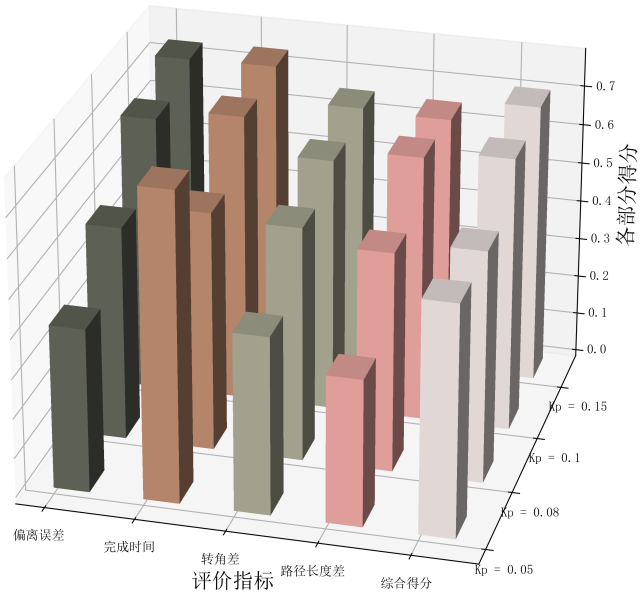


Figure 3.16: Comparison of the overall scores for medium-speed operation

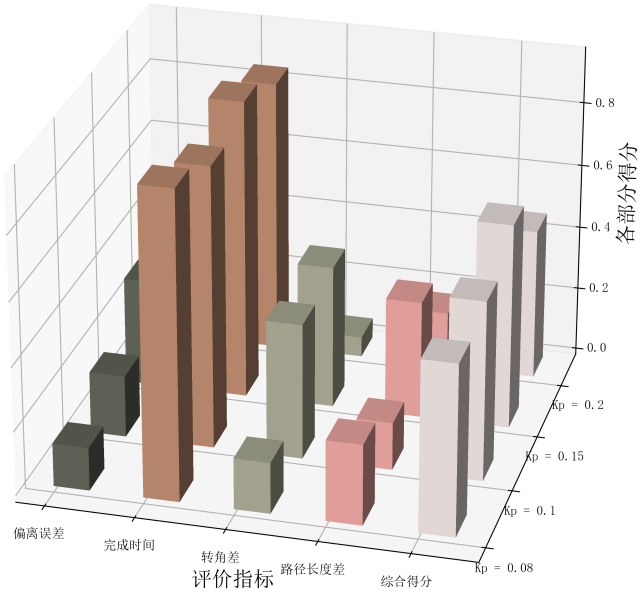


Figure 3.17: Comparison of the overall scores for high-speed operation

Metric		Data			
Low speed	K_p	0.03	0.05	0.08	0.1
	Overall score	0.538	0.575	0.579	0.582
Medium speed	K_p	0.05	0.08	0.1	0.15
	Overall score	0.584	0.585	0.693	0.710
High speed	K_p	0.08	0.1	0.15	0.2
	Overall score	0.532	0.560	0.642	0.471

Table 3.9: Overall scores of the control algorithm without throttle regulation

It can be seen from the figures and the data that the overall performance is best at medium speed, and that the optimal value of K_p differs between speed ranges. In the present table, the optimal value of K_p in the low-speed range lies between about 0.05 and 0.08, the optimal value of K_p in the medium-speed range is about 0.1 to 0.15, and the optimal value of K_p on the high-speed curve is also about 0.15.

In a subsequent manual parameter-tuning process with a step of 0.01, it was found that the performance in the high-speed range is best at $K_p = 0.15$, that the performance in the medium-speed range is best at $K_p = 0.13$, and that at $K_p = 0.1$ the low-speed performance is close to the best score but is not very different from $K_p = 0.09$.

On the basis of these groups of data, and since the factors influencing the manoeuvrability in the simulator are relatively few, it is assumed that the only factor influencing K_p is the ship speed, and a functional curve can be fitted approximately:

$$K_p = 0.2 \times \sqrt{V_{throttle}} - 0.05 \tag{3.2}$$

From Equation 3.2 the recommended value of K_p for each speed can be calculated. For the three speeds considered above this gives:

- Low speed, throttle = 0.5: $K_p = 0.091$;

- Medium speed, throttle = 0.8: $K_p = 0.129$;
- High speed, throttle = 1.0: $K_p = 0.15$;

These values agree with the earlier analysis, so this function was chosen for the calculation of K_p in the subsequent throttle-control optimisation.

3.2.2.2 Results of the Control Algorithm with Throttle Control After the throttle-control optimisation had been applied, five simulator tests were carried out and the average data given in Table 3.10 were obtained. The normalised overall score can also be calculated and compared with the twelve operating conditions considered above, which gives the comparison shown in Figure 3.18.

	Mean deviation	Completion time	Turning-angle difference	Length difference	Overall score
Raw data	0.274	143.913	262.651	2.364	
Normalised value	0.887	0.849	0.850	0.882	0.859

Table 3.10: Results of the control algorithm with the throttle-control optimisation

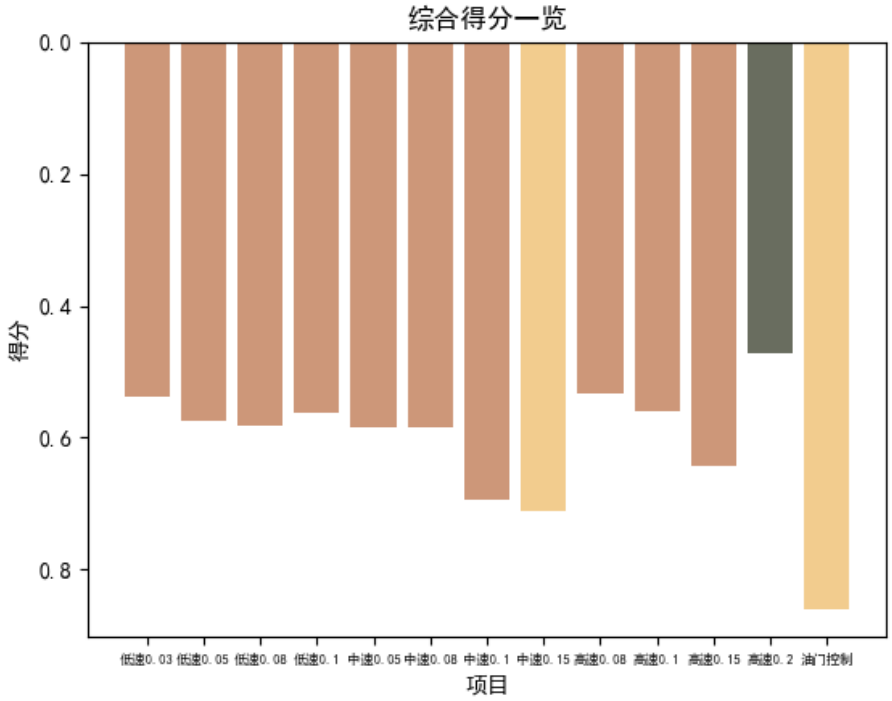


Figure 3.18: Comparison of the overall scores

It can be seen that the algorithm to which the throttle control has been added performs best by far, greatly exceeding the overall scores obtained by the control algorithm with a fixed throttle. There are nevertheless still shortcomings, which are described in detail in Section 4.

4 Remaining Problems

4.1 Problems of the Planning Algorithm

Since the pruning routine in the planning algorithm is rather greedy, when the number of iterations is insufficient the pruned path may become quite strange, as shown in Figure 4.1.

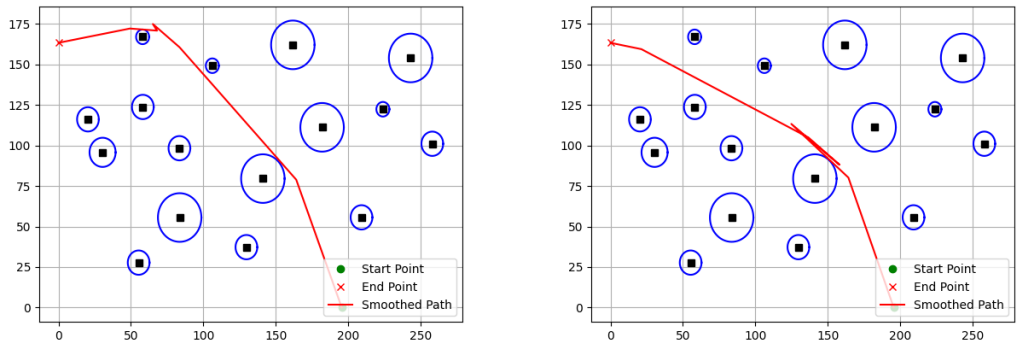


Figure 4.1: Problems of the planning algorithm

The reason for this may be that the path before the RRT pruning already contained some curvature, and the pruning is calculated by a greedy algorithm towards the end of the path, so that the decision point of the pruning was wrong and an obviously incorrect path was generated.

Owing to a lack of time this part has not been solved for the moment and has simply been concealed by the multiple-iteration approach: with 100 iterations this situation basically does not occur.

4.2 Problems of the Control Algorithm

4.2.1 Problems of the Throttle Control

In the throttle-control part, although the parameter K_p is obtained by a function fit and is therefore fairly scientific, the throttle levels within the different ranges were tuned manually and directly. If there is still time later, the best criterion and parameters for the throttle control can be obtained by means of simulator experiments.

4.2.2 Problems of the Data of the Control Group

In the processing of the results of the planning algorithm, the runs were not all completed within the same period of time because of the large number of runs required, while the state of the simulator does fluctuate continuously; as a result, the result of the algorithm with the lowest simulator delay, that is, the throttle control, is far better than the data of the control group.

At the same time, this may also cause an error in the calculated optimal value of K_p.

4.2.3 Problems of the Path of the Planning Algorithm

In the present tests of the planning algorithm only one path was used as the test path for reasons of time, which may lead to the appearance of some errors and makes the final results not entirely rigorous.

5 Conclusions and Outlook

5.1 Summary

Both the planning algorithm and the control algorithm were completed fairly well this time.

In the planning algorithm, after the combined optimisation a nearly perfect path curve could basically be drawn, which laid a very good foundation for the subsequent control algorithm.

In the control algorithm, the LOS-based navigation method with the added throttle-control optimisation can both resist disturbances such as cross winds and waves and achieve the purpose of path-following navigation, and also partly improve various problems in path following such as the speed, the excess turning angle and the deviation from the route.

On the whole, this assignment gave me a deeper understanding of these algorithms and helped me to gain a great deal of valuable experience on different questions such as the coordination between several algorithms and the way in which the quality of an algorithm should be evaluated.

5.2 Outlook

5.2.1 Adding a Perception Module

Yan Xinping et al.[9] proposed the "Navigation Brain" system that serves the navigation of intelligent ships and divided it into a "perception space", a "cognition space" and a "decision and execution space". Drawing on the design concept and the modular design method of the "Navigation Brain" system, this paper divides the model ship test platform into five major modules, namely energy management, perception, decision, execution and communication, and establishes three major monitoring and data-acquisition systems: ship, shore and cloud.

The present algorithm part is mainly concerned with decision making, while perception relies only on GPS signals and inertial navigation; more perception content can be added later.

The safe and efficient control of an intelligent ship depends to a large extent on a suitable navigation system with perception, state estimation, environmental perception and situation awareness capabilities; the perception module provides the platform basis for the testing and verification of these functions. At present, radar, lidar, sonar and various visual sensors have been widely used in the development of perception technology for intelligent ships.[10]

Adding a perception module can better assist the algorithm in making decisions.

5.2.2 Adding Collision-Avoidance Rules

On the basis of the perception module, the ship can be given the ability to avoid collisions: it can perceive and quickly convey the situation around the ship, so that the ship can be guided to perform complex operations, collisions between ships can be avoided, and the ship can be guided to reach its destination smoothly[11].

At present our simulated ships mostly use only the planned path without making changes or avoiding collisions, which means that in the simulation experiments they often hit the ships of other students, so that the experimental data cannot be used.

Adding collision-avoidance rules can greatly improve this aspect and at the same time make the ship more intelligent.

5.2.3 On the Throttle Control

At present the throttle control is related only to the distance and is not linked to the turning angle at the next target point. Later, the range of the throttle control could be changed according to

the size of the turning angle at the target point in order to achieve a better effect.

References

[1] Wenjun Zhang, Yingjun Zhang, and Zihao Liu. Research status and development trend of navigation risk assessment for intelligent ships. *Journal of Dalian Maritime University*, 49(04):13–19, 2023. In Chinese.

[2] Weiming Xu, Rongze Qu, Guoliang Xue, Minyi Xu, Xiao Zhu, Zhijian Liu, and Qiqi Zhang. Research status and development trend of intelligent ship systems. *Ship and Boat*, 34(04):46–55, 2023. In Chinese.

[3] Yanni Sun and Xiaojun Bai. "Made in China 2025": A strategy of a powerful nation with Chinese characteristics. *CAD/CAM and Manufacturing Informatization*, 000(010):43–45, 2020. In Chinese.

[4] Xinping Yan, Jialun Liu, Yu Zhang, Shijie Li, and Tengfei Wang. Research status and prospect of intelligent shipping. *Modern Transportation and Metallurgical Materials*, 2(01):7–18, 2022. In Chinese.

[5] Yongjie Li, Rui Zhang, Muheng Wei, and Yu Zhang. Research status and prospect of key technologies for autonomous ship navigation. *Chinese Journal of Ship Research*, 16(01):32–44, 2021. In Chinese.

[6] Lin Chen and Longxia Yang. Development status of intelligent ships in the major ship-building countries of the world. *Ship Standardization Engineer*, 52(4):5, 2019. In Chinese.

[7] CHH3213. Sampling-based path planning algorithms: Rrt. Website, 2023. https://blog.csdn.net/weixin_42301220/article/details/125254061.

[8] Peng Han, Zhilin Liu, Zecai Zhou, Hao Tang, Liang Ban, and Liulei Hao. Implementation of a track-keeping control algorithm for a self-propelled model based on the los method. *Applied Science and Technology*, 45(3):5, 2018. In Chinese.

[9] Xinping Yan, Chao Wu, and Feng Ma. Concept design of the "navigation brain" for intelligent navigation of cargo ships. *Navigation of China*, 40(4):5, 2017. In Chinese.

[10] Chenghao Han, Jialun Liu, Shijie Li, Yuanchao Hu, and Min Tang. Design and application of a physical test platform for intelligent ship navigation. *Ship Science and Technology*, 46(03):169–173, 2024. In Chinese.

[11] Xiaofeng Xu, Yingjie Xiao, and Xuelai Zhang. Research progress of path optimization for autonomous ships based on collision avoidance regulations. *Journal of Transport Information and Safety*, 39(2):8, 2021. In Chinese.

6 Appendix

Code Listing 7: **Final implementation of the RRT path-planning algorithm**

```
import math
import random
import os
import matplotlib.pyplot as plt
import pandas as pd
import time

map_width = 243
map_height = 167
min_lon = 119.554793879028
max_lon = 119.557477854023
min_lat = 35.403159625348
max_lat = 35.4046581649303


class RRT:
    class Node:
        def __init__(self, x, y):
            self.x = x
            self.y = y
            self.path_x = []
            self.path_y = []
            self.parent = None

    class AreaBounds:
        def __init__(self, area):
            self.xmin =  float(area[0])
            self.xmax =  float(area[1])
            self.ymin =  float(area[2])
            self.ymax =  float(area[3])

    def __init__(self, start, goal, obstacle_list, rand_area, expand_dis=10, goal_sample_rate=5, max_iter=1000,
                 play_area=None, robot_radius=0.0):
        self.start = self.Node(start[0], start[1])
        self.end = self.Node(goal[0], goal[1])
        self.min_rand = rand_area[0]
        self.max_rand = rand_area[1]
        if play_area is not None:
            self.play_area = self.AreaBounds(play_area)
        else:
            self.play_area = None
        self.expand_dis = expand_dis
        self.goal_sample_rate = goal_sample_rate
        self.max_iter = max_iter
        self.obstacle_list = obstacle_list
        self.node_list = []
        self.robot_radius = robot_radius
        self.some_extra_radius = 3.0

    def planning(self, animation=True, camera=None):
        self.node_list = [self.start]
        valid_path = None

        for i in  range(self.max_iter):
            rnd_node = self.sample_free()

            nearest_ind = self.get_nearest_node_index(self.node_list, rnd_node)
            nearest_node = self.node_list[nearest_ind]

            new_node = self.steer(nearest_node, rnd_node, self.expand_dis)

            if self.is_inside_play_area(new_node, self.play_area) and self.obstacle_free(new_node, self.
                 obstacle_list, self.robot_radius):
                self.node_list.append(new_node)

            if self.calc_dist_to_goal(self.node_list[-1].x, self.node_list[-1].y) <= (
                    self.expand_dis + self.some_extra_radius):
                final_node = self.steer(self.node_list[-1], self.end, self.expand_dis)
```

```
                if self.obstacle_free(final_node, self.obstacle_list, self.robot_radius):
                    valid_path = self.generate_final_course( len(self.node_list) - 1)
                    break

            if animation and i % 5 == 0:
                self.draw_graph(rnd_node, camera)

        if valid_path is not None:
            final_path_nodes = [self.Node(x, y) for (x, y) in valid_path]
            smoothed_path = self.smooth_path(final_path_nodes)
            valid_path = [[node.x, node.y] for node in smoothed_path]

        return valid_path

    def steer(self, from_node, to_node, extend_length= float("inf")):
        d, theta = self.calc_distance_and_angle(from_node, to_node)
        max_angle_diff = math.radians(30)
        if from_node.parent is not None:
            parent_angle = math.atan2(from_node.y - from_node.parent.y, from_node.x - from_node.parent.x)
            angle_diff =  abs(theta - parent_angle)
            if angle_diff > max_angle_diff:
                theta = parent_angle + max_angle_diff if theta > parent_angle else parent_angle - max_angle_diff

        if extend_length >= d:
            new_x = to_node.x
            new_y = to_node.y
        else:
            new_x = from_node.x + math.cos(theta) * extend_length
            new_y = from_node.y + math.sin(theta) * extend_length

        new_node = self.Node(new_x, new_y)
        new_node.path_x = [from_node.x]
        new_node.path_y = [from_node.y]
        new_node.path_x.append(new_x)
        new_node.path_y.append(new_y)

        new_node.parent = from_node

        return new_node

    def generate_final_course(self, goal_ind):
        path = [[self.end.x, self.end.y]]
        node = self.node_list[goal_ind]
        while node.parent is not None:
            path.append([node.x, node.y])
            node = node.parent
        path.append([node.x, node.y])

        return path

    def smooth_path(self, path):
        pruned_path = [path[0]]
        i = 0
        while i <  len(path) - 1:
            # Look for the next pruning point
            j = i + 1
            while j <  len(path) and self.line_collision_free(pruned_path[-1], path[j], self.obstacle_list):
                j += 1
            pruned_path.append(path[j-1])
            i += 1

        # Append the goal point to the smoothed path
        pruned_path.append(path[-1])
        unique_path = [pruned_path[-1]]
        for point in  reversed(pruned_path[:-1]):
            if point not in unique_path:
                unique_path.insert(0, point)

        return unique_path
    @staticmethod
    def line_collision_free(node1, node2, obstacle_list):
        for (ox, oy, size) in obstacle_list:
            # Compute the relative position of the path segment with respect to the circle
```

```
            dx = node2.x - node1.x
            dy = node2.y - node1.y
            d = math.sqrt(dx ** 2 + dy ** 2) + 10 ** (-8)
            t = ((ox - node1.x) * dx + (oy - node1.y) * dy) / (d ** 2)
            t =  max(0,  min(1, t))  # Make sure that t lies within [0, 1]

            # Compute the point of the path segment closest to the centre of the circle
            closest_x = node1.x + t * dx
            closest_y = node1.y + t * dy

            # The segment intersects the circle if the closest point is within the radius
            dist = math.sqrt((closest_x - ox) ** 2 + (closest_y - oy) ** 2)
            if dist <= size:
                return False

        return True

    def calc_dist_to_goal(self, x, y):
        dx = x - self.end.x
        dy = y - self.end.y
        return math.hypot(dx, dy)

    def sample_free(self):
        if random.randint(0, 100) > self.goal_sample_rate:
            rnd = self.Node(
                random.uniform(self.min_rand, self.max_rand),
                random.uniform(self.min_rand, self.max_rand)
            )
        else:
            rnd = self.Node(self.end.x, self.end.y)

        return rnd

    def draw_graph(self, rnd=None, camera=None):
        if camera is None:
            plt.clf()

        plt.gcf().canvas.mpl_connect('key_release_event', lambda event: [exit(0) if event.key == 'escape' else
             None])

        for node in self.node_list:
            if node.parent:
                plt.plot(node.path_x, node.path_y, "-g")

        for (ox, oy, size) in self.obstacle_list:
            self.plot_circle(ox, oy, size)

        if self.play_area is not None:
            plt.plot([self.play_area.xmin, self.play_area.xmax, self.play_area.xmax, self.play_area.xmin,
                      self.play_area.xmin],
                     [self.play_area.ymin, self.play_area.ymin, self.play_area.ymax, self.play_area.ymax,
                      self.play_area.ymin], "-k")

        all_x = [node.x for node in self.node_list]
        all_y = [node.y for node in self.node_list]
        all_x.extend([ox for (ox, oy, size) in self.obstacle_list])
        all_y.extend([oy for (ox, oy, size) in self.obstacle_list])

        if self.play_area is not None:
            all_x.extend([self.play_area.xmin, self.play_area.xmax])
            all_y.extend([self.play_area.ymin, self.play_area.ymax])

        plt.axis("equal")
        plt.axis([ min(all_x) - 2,  max(all_x) + 2,  min(all_y) - 2,  max(all_y) + 2])

        plt.grid(True)
        plt.pause(0.01)

    @staticmethod
    def plot_circle(x, y, size, color="-b"):
        deg =  list( range(0, 360, 5))
        deg.append(0)
        xl = [x + size * math.cos(math.radians(d)) for d in deg]
```

```
213         yl = [y + size * math.sin(math.radians(d)) for d in deg]
214         plt.plot(xl, yl, color)
215 
216     @staticmethod
217     def get_nearest_node_index(node_list, rnd_node):
218         dlist = [(node.x - rnd_node.x) ** 2 + (node.y - rnd_node.y) ** 2 for node in node_list]
219         minind = dlist.index( min(dlist))
220 
221         return minind
222 
223     @staticmethod
224     def is_inside_play_area(node, play_area):
225         if play_area is None:
226             return True
227 
228         if node.x < play_area.xmin or node.x > play_area.xmax or node.y < play_area.ymin or node.y > play_area.
                ymax:
229             return False
230         else:
231             return True
232 
233     @staticmethod
234     def obstacle_free(node, obstacle_list, robot_radius):
235         if node is None:
236             return False
237 
238         for (ox, oy, size) in obstacle_list:
239             dx_list = [ox - x for x in node.path_x]
240             dy_list = [oy - y for y in node.path_y]
241             d_list = [dx * dx + dy * dy for (dx, dy) in  zip(dx_list, dy_list)]
242 
243             if  min(d_list) <= (size + robot_radius) ** 2:
244                 return False
245 
246         return True
247 
248     @staticmethod
249     def calc_distance_and_angle(from_node, to_node):
250         dx = to_node.x - from_node.x
251         dy = to_node.y - from_node.y
252         d = math.hypot(dx, dy)
253         theta = math.atan2(dy, dx)
254         return d, theta
255 
256 def visualize_result(rrt, shortest_path):
257     fig, ax = plt.subplots()
258     #rrt.draw_graph(rnd=None, camera=ax)
259     plt.plot([x for (x, y, _) in rrt.obstacle_list], [y for (x, y, _) in rrt.obstacle_list], 'sk')
260     for (ox, oy, size) in rrt.obstacle_list:
261         rrt.plot_circle(ox, oy, size)
262     plt.plot(rrt.start.x, rrt.start.y, 'go', label='Start Point')
263     plt.plot(rrt.end.x, rrt.end.y, 'rx', label='End Point')
264     # Make sure that the start point is contained in the final path
265     plt.plot([node.x for node in shortest_path], [node.y for node in shortest_path], '-r', label='Smoothed Path')
266     plt.grid(True)
267     plt.legend()
268     plt.savefig('final_result_shortest_path.png')
269     plt.show()
270 
271     # Output the path coordinates
272     Fpath = output_path(shortest_path)
273     print(Fpath)
274 
275 def calculate_path_turning(path):
276     total_turning = 0.0
277     for i in  range(1,  len(path) - 1):
278         prev_node = path[i - 1]
279         current_node = path[i]
280         next_node = path[i + 1]
281 
282         angle1 = math.atan2(current_node.y - prev_node.y, current_node.x - prev_node.x)
283         angle2 = math.atan2(next_node.y - current_node.y, next_node.x - current_node.x)
284 
```

```
        turning =  abs(angle2 - angle1)
        total_turning += turning

    return total_turning


def run_rrt():
    # Initialise the map
    start_x, start_y, goal_x, goal_y, obstacle_list = initialize_map()
    start_time = time.time()  # Record the start time
    shortest_path = None
    shortest_length =  float('inf')
    path_lengths = []
    path_turnings = []
    path_numbers = []
    path_score = []
    range_number = 200
    sum_number = 0

    # Carry out the path planning
    for _ in  range(range_number):
        rrt = RRT(
            start=[start_x, start_y],
            goal=[goal_x, goal_y],
            rand_area=[0, 250],
            obstacle_list=obstacle_list,
            play_area=[0, 300, 0, 200],
            robot_radius=4.0
        )

        valid_path = rrt.planning(animation=False, camera=None)

        # If a valid path has been found, update the shortest path
        if valid_path is not None:
            sum_number += 1
            final_path_nodes = [rrt.Node(x, y) for (x, y) in valid_path]
            smoothed_path = rrt.smooth_path(final_path_nodes)
            path_length = calculate_path_length(smoothed_path)
            path_lengths.append(path_length)
            path_turning = calculate_path_turning(smoothed_path) * 180 / math.pi
            path_turnings.append(path_turning)
            path_numbers.append( len(smoothed_path))
            Path_score = 0.47 * ((358.78 - path_length) / 104.07) + 0.28 * ((1500 - path_turning) /1500) + 0.18 *
                  ((42 -
                 len(smoothed_path)) / 40)
            path_score.append(Path_score)
            if path_length < shortest_length:
                shortest_length = path_length
                shortest_path = smoothed_path


    # If the shortest path has been found, visualise and output it
    if shortest_path is not None:
        visualize_result(rrt, shortest_path)


    end_time = time.time()  # Record the end time
    total_time = end_time - start_time  # Compute the total time

    # Write the data to an Excel sheet
    data = {
        'Path Length': path_lengths,
        'Path Turning': path_turnings,
        'Path Number': path_numbers,
        'Path Score': path_score,
    }
    df = pd.DataFrame(data)
    df.to_excel('path_data.xlsx', index=False)

    # Output the average values
    avg_path_length =  sum(path_lengths) / sum_number
    avg_path_turning =  sum(path_turnings) / sum_number
    avg_path_number =  sum(path_numbers) / sum_number
    avg_Fpath_length = shortest_length
```

```
357     avg_Fpath_turning = calculate_path_turning(shortest_path) * 180 / math.pi
358     avg_Fpath_number =  len(shortest_path)
359     Fpath_score = 0.47 * ((358.78 - avg_Fpath_length) / 104.07) + 0.28 * ((1500 - avg_Fpath_turning) / 1500) +
            0.18 * (
360                 (42 - avg_Fpath_number)/ 40) + 0.07 * ((15 - total_time) / 15)
361     print("Average Path Length:", avg_path_length)
362     print("Average Path Turning:", avg_path_turning)
363     print("Average Path Number:", avg_path_number)
364     print("Fpath Length:", avg_Fpath_length)
365     print("Fpath Turning:", avg_Fpath_turning)
366     print("Fpath Number:", avg_Fpath_number)
367     print("Totle Time:", total_time)
368     print("Fpath Score:", Fpath_score)
369
370
371 def initialize_map():
372
373     goal_lon, goal_lat = 119.554793879028, 35.404625176547
374     start_lon, start_lat = 119.556952728385, 35.403159625348
375     obstacle_list_lon_lat = [
376         [119.555018460937, 35.4042015549788, 7.5],
377         [119.555129981354, 35.404018856646, 9.0],
378         [119.555436799912, 35.404270055301, 7.5],
379         [119.555715656054, 35.40404166038, 7.5],
380         [119.555435148977, 35.4046581649303, 4.5],
381         [119.555966549108, 35.4044983605518, 4.5],
382         [119.556580076699, 35.4046138199964, 15.0],
383         [119.555717527382, 35.4036580568152, 15.0],
384         [119.557477854023, 35.4045413532812, 15.0],
385         [119.556351640929, 35.4038734732142, 15.0],
386         [119.556803007337, 35.4041573899354, 15.0],
387         [119.555404873774, 35.4034083908053, 7.5],
388         [119.556226139639, 35.4034940164831, 7.5],
389         [119.557103550729, 35.4036589584682, 7.5],
390         [119.557642987829, 35.4040663565401, 7.5],
391         [119.557266482617, 35.40425743656, 4.5]
392     ]
393
394     start_x, start_y = convert_to_map_coordinate(start_lon, start_lat)
395     goal_x, goal_y = convert_to_map_coordinate(goal_lon, goal_lat)
396     obstacle_list = [[convert_to_map_coordinate(lon, lat)[0], convert_to_map_coordinate(lon, lat)[1], size] for
            lon,
397                      lat, size in obstacle_list_lon_lat]
398
399     #print(start_x, start_y, goal_x, goal_y)
400
401     return start_x, start_y, goal_x, goal_y, obstacle_list
402
403 def convert_to_map_coordinate(lon, lat):
404     x = (lon - min_lon) / (max_lon - min_lon) * map_width
405     y = (lat - min_lat) / (max_lat - min_lat) * map_height
406     return x, y
407
408 def convert_to_lon_lat(x, y):
409     lon = x / map_width * (max_lon - min_lon) + min_lon
410     lat = y / map_height * (max_lat - min_lat) + min_lat
411     return lon, lat
412
413 def output_path(path):
414     print("path = [")
415     Fpath = []
416     for node in path:
417         lon, lat = convert_to_lon_lat(node.x, node.y)
418         Fpath.append({'longitude': lon, 'latitude': lat})
419         print(f"    {{'longitude': {lon}, 'latitude': {lat}}},")
420     print("]")
421     return Fpath
422
423 def calculate_path_length(path):
424     total_length = 0.0
425     for i in  range( len(path) - 1):
426         node1 = path[i]
427         node2 = path[i + 1]
```

```
428             length = math.hypot(node2.x - node1.x, node2.y - node1.y)
429             total_length += length
430         return total_length
431 
432 def main():
433     run_rrt()
434 
435 if __name__ == '__main__':
436     main()
```

Code Listing 8: **Final implementation of the LOS and PID path-tracking algorithm**

```
import socket
import json
import threading
import time
import pandas as pd
import math

Host = '127.0.0.1'  # IP address

Port_GPS = 60100  # GPS port
Port_IMU = 60101  # IMU port
Port_Buoy = 60103  # buoy port

Port_Control = 60199  # control port
Port_Path = 60198  # path-output port

socket_GPS = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)  # Create the UDP sockets
socket_IMU = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
socket_Buoy = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

socket_Control = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
socket_Path = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

socket_GPS.bind((Host, Port_GPS))  # Bind the ports
socket_IMU.bind((Host, Port_IMU))
socket_Buoy.bind((Host, Port_Buoy))

longitude = None  # longitude
latitude = None  # latitude

pitch = None  # pitch angle
roll = None  # roll angle
yaw = None  # yaw angle
velocity = None  # velocity

buoy_list = None  # buoy positions

path = None  # global path variable

pre_error = 0
sum_error = 0

def GPS_receive():
    global longitude, latitude
    while True:
        data, _ = socket_GPS.recvfrom(1024)
        data_json = json.loads(data.decode('utf-8'))
        longitude = data_json["longitude"]
        latitude = data_json["latitude"]

def IMU_receive():
    global pitch, roll, yaw, velocity
    while True:
        data, _ = socket_IMU.recvfrom(1024)
        data_json = json.loads(data.decode('utf-8'))
        pitch = data_json["pitch"]
        roll = data_json["roll"]
        yaw = data_json["yaw"]
        velocity = data_json["velocity"]

def Buoy_receive():
    global start, goal, obstacle_list
    while True:
        data, _ = socket_Buoy.recvfrom(10240)
        data_json = json.loads(data.decode('utf-8'))

        start =  list(data_json['start'].values())
        goal =  list(data_json['end'].values())
        obstacle_list = []
        for i in  range( len(data_json['obstacles'])):
            obstacle_list.append( list(data_json['obstacles'][i].values()))

```

```python
def Path_planning():
    global path
    path = [{'longitude': 119.555323713895, 'latitude': 35.4059490670712}, {'longitude': 119.554793879028, '
         latitude': 35.404625176547}, {'longitude': 119.5561758920993, 'latitude': 35.40428068759785}, {'
         longitude': 119.55642159573607, 'latitude': 35.40409235970548}, {'longitude': 119.55656392206485, '
         latitude': 35.40389586868287}, {'longitude': 119.556952728385, 'latitude': 35.403159625348}]

def Path_send(path_points):
    path = {"points": path_points}
    path = json.dumps(path)
    path = path.encode('utf-8')
    socket_Path.sendto(path, (Host, Port_Path))

def calculate_distance(lat1, lon1, lat2, lon2):
    R = 6371.0
    lat1 = math.radians(lat1)
    lon1 = math.radians(lon1)
    lat2 = math.radians(lat2)
    lon2 = math.radians(lon2)

    dlon = lon2 - lon1
    dlat = lat2 - lat1

    a = math.sin(dlat / 2)**2 + math.cos(lat1) * math.cos(lat2) * math.sin(dlon / 2)**2
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

    distance = R * c

    return distance

def calculate_angle(lat1, lon1, lat2, lon2):
    lat1 = math.radians(lat1)
    lon1 = math.radians(lon1)
    lat2 = math.radians(lat2)
    lon2 = math.radians(lon2)

    dlon = lon2 - lon1
    y = math.sin(dlon) * math.cos(lat2)
    x = math.cos(lat1) * math.sin(lat2) - math.sin(lat1) * math.cos(lat2) * math.cos(dlon)
    angle = math.atan2(y, x)

    angle = math.degrees(angle)

    return angle

def Model():
    global path
    global path_index
    path_index = 0
    total_distance = 0  # Total distance travelled by the ship
    total_angle = 0  # Total angle swept by the bow of the ship
    prev_yaw = yaw
    perpendicular_distances = []  # Length of the perpendicular from the hull to the path
    start_time = time.time()  # Time at which the ship started
    end_time = None  # Time at which the ship reached the goal
    prev_position = {'latitude': None, 'longitude': None}  # Previous position of the hull
    angles = []
    distances = []
    indexs = []
    Path_send(path)

    while True:
        if longitude is None or latitude is None or pitch is None or roll is None or yaw is None:
            time.sleep(0.1)
            continue

        target_yaw, throttle, x_per, y_per = LOS({'latitude': latitude, 'longitude': longitude}, path[path_index
             ])

        steer = PID_controller(yaw, target_yaw, throttle)

        if path_index >=  len(path) :
            throttle = 0
```

```
            steer = 0
            print("Goal point reached")
            Control_send(throttle, steer)
            end_time = time.time()  # Time at which the ship reached the goal
            break  # Leave the loop

        Control_send(throttle, steer)


        # Compute the distance travelled by the ship and add it to the total length
        if prev_position['latitude'] is not None and prev_position['longitude'] is not None:
            distance = calculate_distance(prev_position['latitude'], prev_position['longitude'], latitude,
                 longitude)
            if path_index > 1:
                total_distance += distance
                distances.append(distance)

        # Compute the heading angle of the ship and add it to the total angle
        if prev_yaw is not None:
            angle =  abs(prev_yaw - yaw)
            if path_index > 1:
                total_angle +=  abs(angle)
                angles.append(angle)

        # Compute the length of the perpendicular from the hull to the path and store it

        perpendicular_distance = calculate_distance(latitude, longitude, x_per, y_per)
        if perpendicular_distance is not None:
            if path_index > 1:
                perpendicular_distances.append(perpendicular_distance)
                indexs.append(path_index + 1)

        prev_position['latitude'] = latitude
        prev_position['longitude'] = longitude
        prev_yaw = yaw
        #print(latitude, longitude)
        time.sleep(0.1)
        #print(total_distance)
    data = {
         'Path_Index:': indexs,
         #'Distances': distances,
         #'Angles': angles,
         'Perpendicular_distances': perpendicular_distances,
    }
    df = pd.DataFrame(data)
    df.to_excel('path_data.xlsx', index=False)
    print( len(angles),  len(perpendicular_distances),  len(distances),  len(indexs))
    print("Total distance travelled by the ship:", total_distance * 1000, "m")
    print("Total angle swept by the bow of the ship:", total_angle, "deg")
    print("Mean length of the perpendicular from the hull to the path:", 1000 *  sum(perpendicular_distances)/
         len(perpendicular_distances))

    # Compute the time the ship needs to travel from the start to the goal
    if end_time is not None:
        time_taken = end_time - start_time
        print("Time from the start point to the goal point:", time_taken, "s")
    else:
        print("The travel time cannot be computed because the goal point was not reached")

def perpendicular_distance_to_path(position, point1, point2):
    # Compute the perpendicular distance from the position of the hull to the path
    lat1, lon1 = position['latitude'], position['longitude']
    lat2, lon2 = point1['latitude'], point1['longitude']
    lat3, lon3 = point2['latitude'], point2['longitude']

    # Length of the segment
    line_length = calculate_distance(lat2, lon2, lat3, lon3)

    # If the segment has zero length, return the point-to-point distance
    if line_length == 0:
        return calculate_distance(lat1, lon1, lat2, lon2)

    # Compute the unit vector of the segment
    dx = (lon3 - lon2) / line_length
```

```
    dy = (lat3 - lat2) / line_length

    # Compute the coordinates of the perpendicular distance
    x = (lon1 - lon2) * dx + (lat1 - lat2) * dy
    y = (lat1 - lat2) * dx - (lon1 - lon2) * dy

    # The length of the perpendicular from the hull to the path is the absolute value of y
    return  abs(y)

def LOS(current_position, target_point, Delta = 5):
    global path_index
    global sum_error
    global pre_error
    distance_per_degree_lat = 128320
    distance_per_degree_lon = 100000

    def calculate_distance(lat1, lon1, lat2, lon2):
        R = 6371.0
        lat1 = math.radians(lat1)
        lon1 = math.radians(lon1)
        lat2 = math.radians(lat2)
        lon2 = math.radians(lon2)

        dlon = lon2 - lon1
        dlat = lat2 - lat1

        a = math.sin(dlat / 2)**2 + math.cos(lat1) * math.cos(lat2) * math.sin(dlon / 2)**2
        c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

        distance = R * c

        return distance

    def calculate_angle(lat1, lon1, lat2, lon2):
        lat1 = math.radians(lat1)
        lon1 = math.radians(lon1)
        lat2 = math.radians(lat2)
        lon2 = math.radians(lon2)

        dlon = lon2 - lon1
        y = math.sin(dlon) * math.cos(lat2)
        x = math.cos(lat1) * math.sin(lat2) - math.sin(lat1) * math.cos(lat2) * math.cos(dlon)
        angle = math.atan2(y, x)

        angle = math.degrees(angle)

        return angle

    distance_to_target = calculate_distance(current_position['latitude'], current_position['longitude'],
                                            target_point['latitude'], target_point['longitude'])
    target_point_next = path[path_index - 1]
    distance_to_target_next = calculate_distance(current_position['latitude'], current_position['longitude'],
         target_point_next['latitude'], target_point_next['longitude'])
    if distance_to_target < 0.005:
        if path_index >=  len(path) -1:
            if distance_to_target < 0.0035:
                path_index += 1
                return 0, 0, current_position['latitude'], current_position['longitude']
        if path_index <  len(path) - 1:
            path_index += 1
            target_point = path[path_index]
            target_point_next = path[path_index - 1]
    #print(path_index)
    if distance_to_target < 0.005 or distance_to_target_next < 0.005:
        throttle = 0.4
    if distance_to_target > 0.005 and distance_to_target < 0.02 and distance_to_target_next > 0.005:
        throttle = 0.8
    if distance_to_target > 0.02 and distance_to_target_next > 0.005:
        throttle = 1.0
    #throttle = 1

    #print("target_point:", target_point)
    #print("target_point_next:", target_point_next)
```

```
286     if target_point_next['latitude'] != target_point['latitude']:
287         k = (target_point_next['longitude'] - target_point['longitude']) / (target_point_next['latitude'] -
                target_point['latitude'])
288         b = target_point_next['longitude'] - k * target_point_next['latitude']
289         x_perpendicular = (current_position['latitude'] + k * (current_position['longitude'] - b)) / (1 + k ** 2)
290         y_perpendicular = k * x_perpendicular + b
291         direction_angle = calculate_angle(x_perpendicular, y_perpendicular, target_point['latitude'],
                target_point['longitude'])
292         delta_latitude = (Delta / distance_per_degree_lat) * math.cos(math.radians(direction_angle))
293         delta_longitude = (Delta / distance_per_degree_lon) * math.sin(math.radians(direction_angle))
294         #print("x_perpendicular:" + str(x_perpendicular) + " y_perpendicular:" + str(y_perpendicular))
295         #print("delta_latitude:" + str(delta_latitude) + " delta_longitude:" + str(delta_longitude))
296 
297         new_latitude = x_perpendicular + delta_latitude
298         new_longitude = y_perpendicular + delta_longitude
299         DotLos = {'latitude': new_latitude, 'longitude':new_longitude}
300     else:
301         DotLos = {'latitude': target_point['latitude'],
302                   'longitude': target_point['longitude']}
303 
304     angle = calculate_angle(current_position['latitude'], current_position['longitude'],
305                             DotLos['latitude'], DotLos['longitude'])
306     #print(DotLos)
307     #print(angle)
308     return angle, throttle, x_perpendicular, y_perpendicular
309 
310 def PID_controller(current_yaw, target_yaw, throttle):
311     global pre_error, sum_error
312     yaw_difference = target_yaw - current_yaw
313 
314     if  abs(yaw_difference) > 180:
315         if yaw_difference > 0:
316             error = yaw_difference - 360
317         else:
318             error = yaw_difference + 360
319     else:
320         error = yaw_difference
321 
322     sum_error += error * 0.1
323     d_error = error - pre_error
324     Kp = 0.15 * math.sqrt(throttle)
325     Ki = 0
326     Kd = 0 * math.sqrt(throttle)
327     steer = Kp * error + Ki * sum_error + Kd * d_error
328 
329     steer =  max( min(steer, 1), -1)
330 
331     pre_error = error
332     #print(error)
333     #print('steer: ', steer)
334 
335     return steer
336 
337 def Control_send(throttle, steer):
338     control_cmd = {}
339     control_cmd["throttle"] = throttle
340     control_cmd["steer"] = steer
341     #print(control_cmd)
342     control_cmd = json.dumps(control_cmd)
343     control_cmd = control_cmd.encode('utf-8')
344     socket_Control.sendto(control_cmd, (Host, Port_Control))
345 
346 if __name__ == "__main__":
347     # Multi-threaded reception
348     thread_GPS = threading.Thread(target=GPS_receive, daemon=True)
349     thread_IMU = threading.Thread(target=IMU_receive, daemon=True)
350     thread_Buoy = threading.Thread(target=Buoy_receive, daemon=True)
351     thread_GPS.start()
352     thread_IMU.start()
353     thread_Buoy.start()
354 
355     # Path planning
356     Path_planning()
```

```
357
358     # Control program
359     Model()
```